Software Engineering

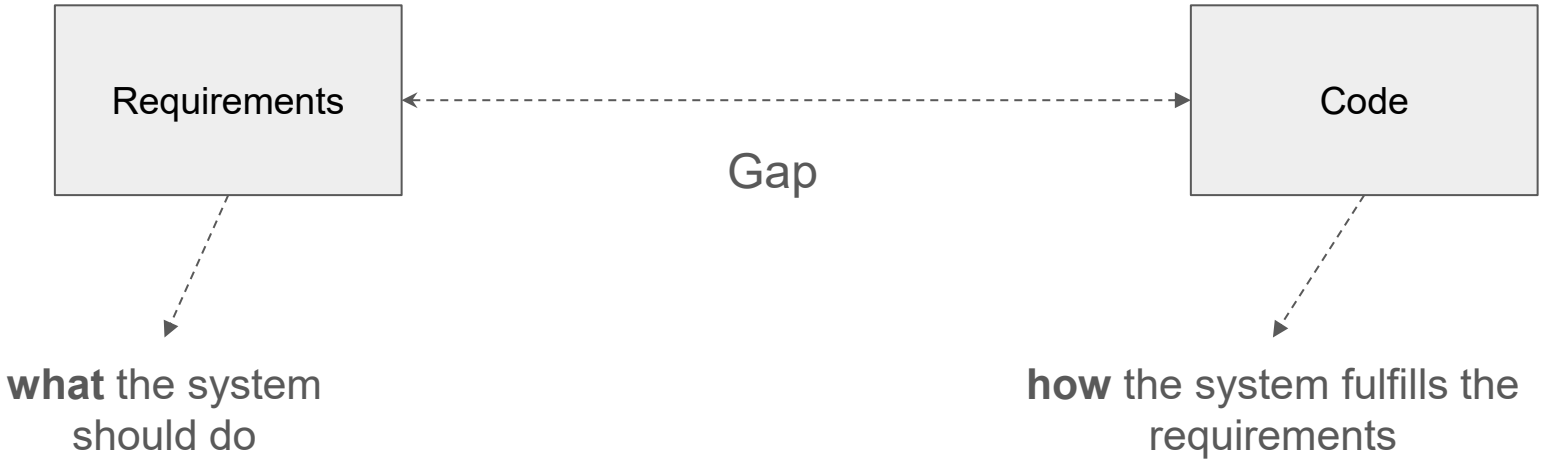
Behavior Modeling using UML

Outline

- Package Diagram
- Behavior Modeling
- State Machine Diagram
- Activity Diagram
- Sequence Diagram
- Communication (collaboration) Diagram

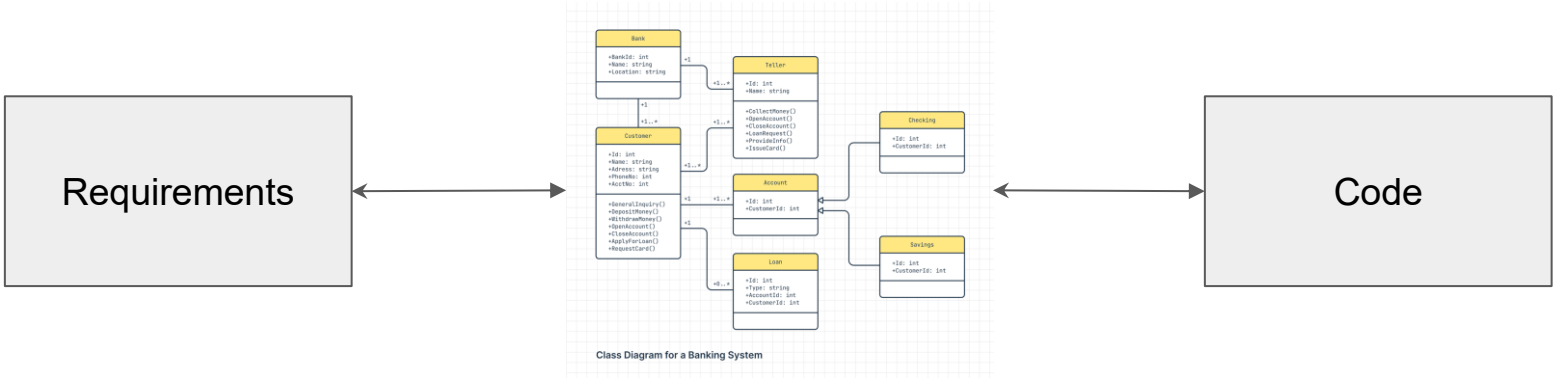
From previous lecture

- There is a gap between these two domains: requirements and code



Software Models

- Goal: bridge the gap between requirements and code
- Models document solutions to problems defined by requirements



UML

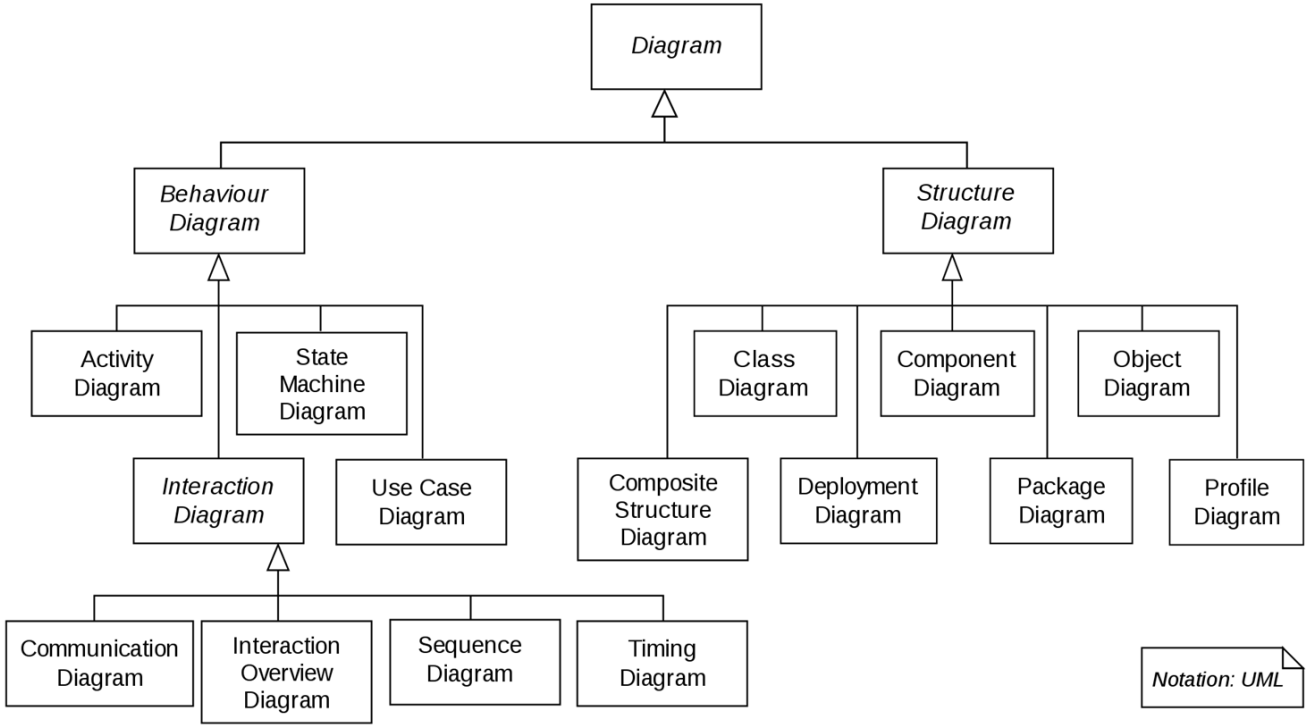
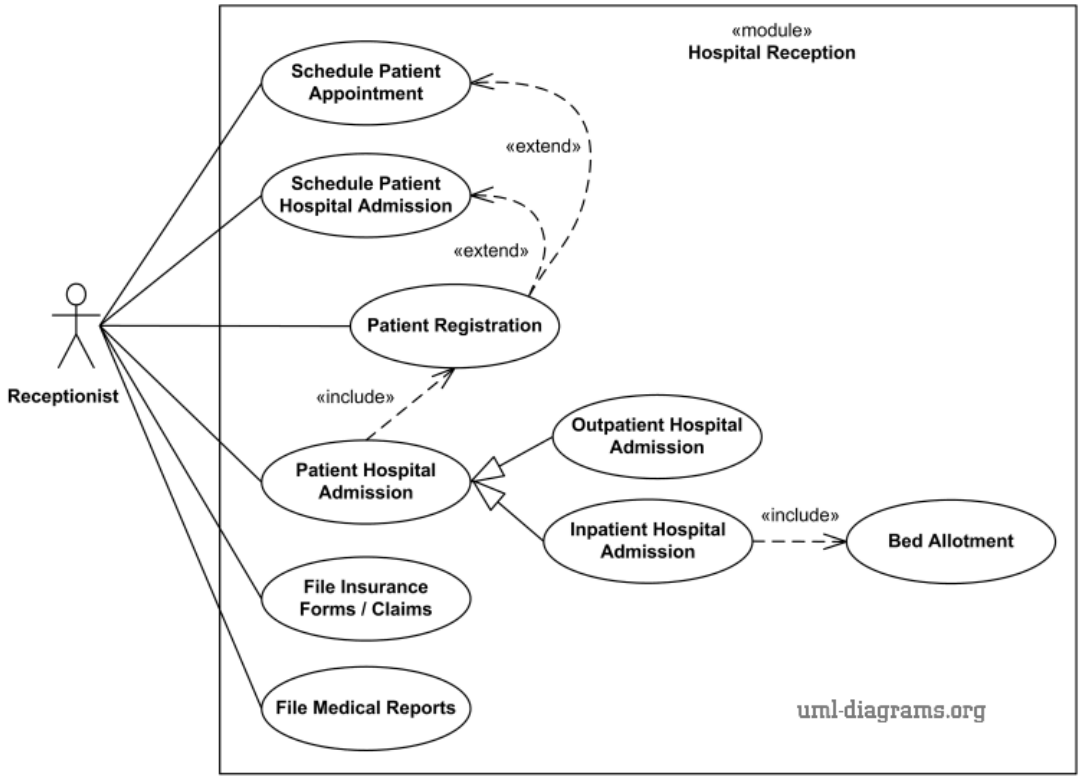


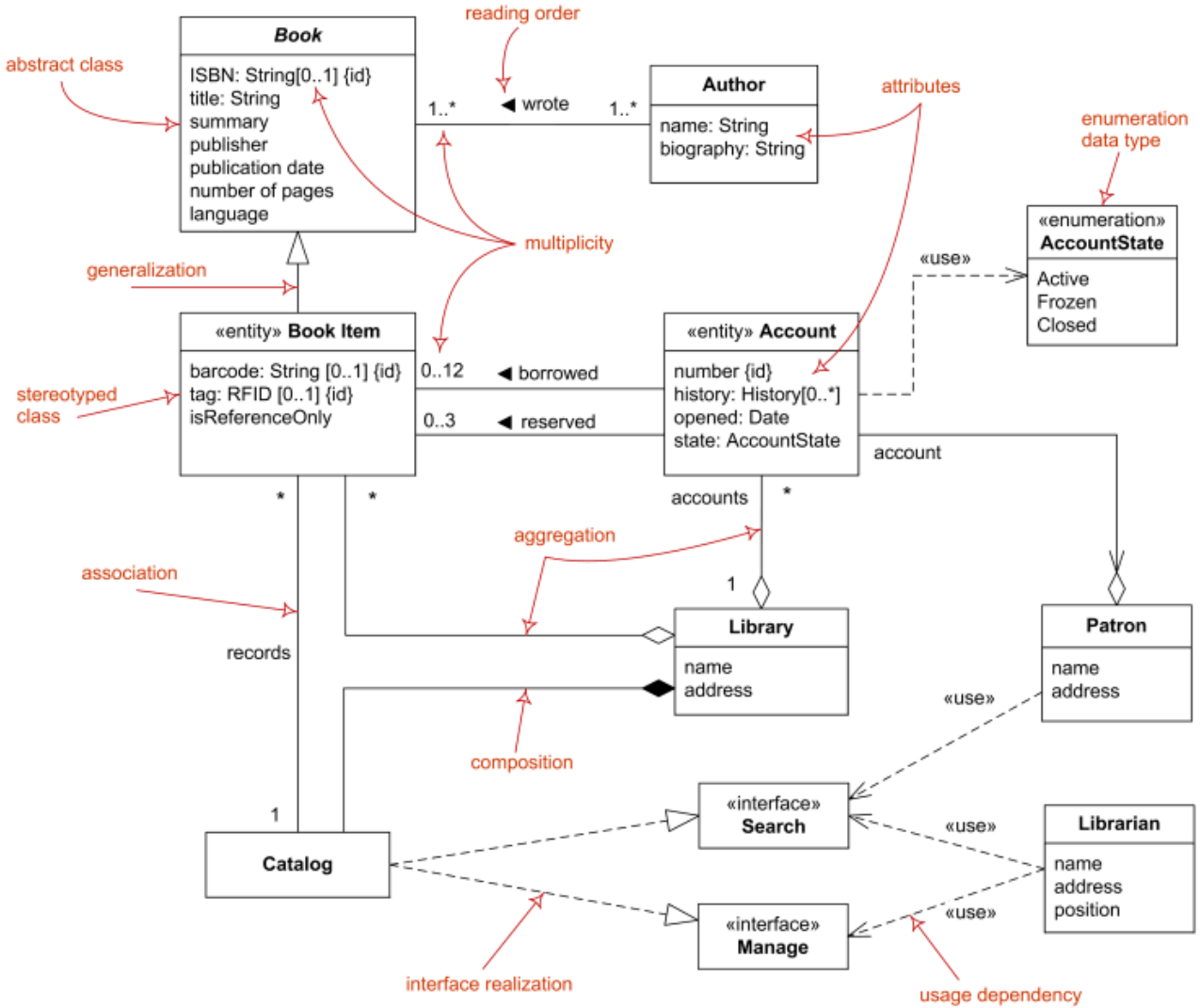
Diagram
Behaviour Diagram
Structure Diagram
Activity Diagram
State Machine Diagram
Interaction Diagram
Use Case Diagram
Class Diagram
Component Diagram
Object Diagram
Composite Structure Diagram
Deployment Diagram
Package Diagram
Profile Diagram
Communication Diagram
Interaction Overview Diagram
Sequence Diagram
Timing Diagram
Notation: UML

Use Cases Diagram

Use Cases Diagram - Example



Class Diagrams


Book
ISBN: String[0..1] {id}
title: String
summary
publisher
publication date
number of pages
language
abstract class
reading order
1..*
wrote
1..*
Author
name: String
biography: String
attributes
enumeration
data type
«enumeration»
AccountState
Active
Frozen
Closed
«use»
multiplicity
generalization
«entity» Book Item
barcode: String [0..1] {id}
tag: RFID [0..1] {id}
isReferenceOnly
stereotyped
class
0..12
borrowed
0..3
reserved
«entity» Account
number {id}
history: History[0..*]
opened: Date
state: AccountState
account
accounts
*
*
*
aggregation
association
1
Library
name
address
Patron
name
address
records
composition
«use»
1
«interface»
Search
«use»
Librarian
name
address
position
Catalog
«interface»
Manage
«use»
interface realization
usage dependency

Package Diagrams

Package Diagram

- A structural UML diagram
- Shows organization of model elements
- Represents packages and dependencies

Package Diagrams

Purpose

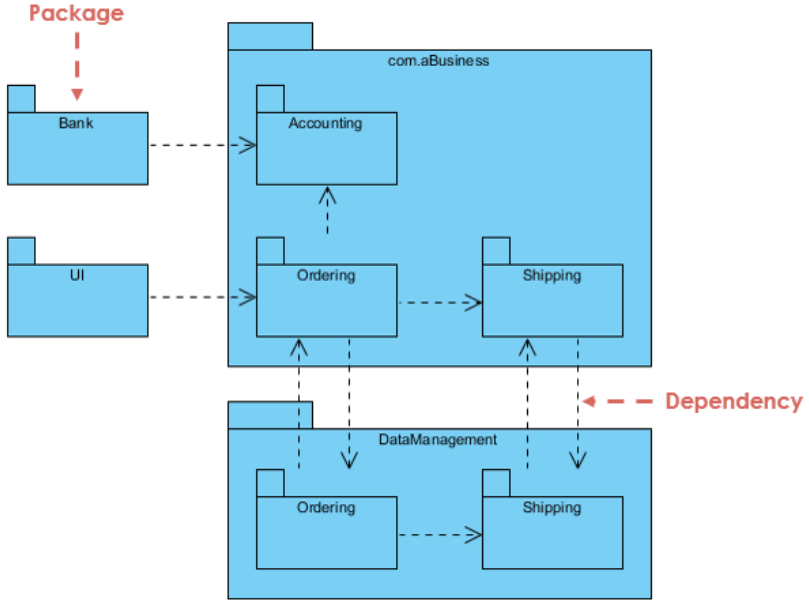
- Organize large systems
- Group related classes
- Reduce complexity
- Clarify high-level architecture

Package Diagrams : Elements

- Package: logical grouping of elements (a container for various elements like classes and interfaces)
- Classes (inside packages)
- Dependencies: arrows showing relationships
- Import / Access relationships

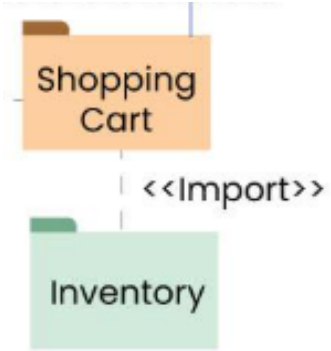
Package Diagrams : Types of Dependencies

- **Dependency**: Dependencies indicate that changes in one package may affect another. This relationship signifies that one element or package relies on another, highlighting how interconnected they are.



Package Diagrams : Types of Dependencies

- «import»: signifies that one package imports the functionality or elements of another package

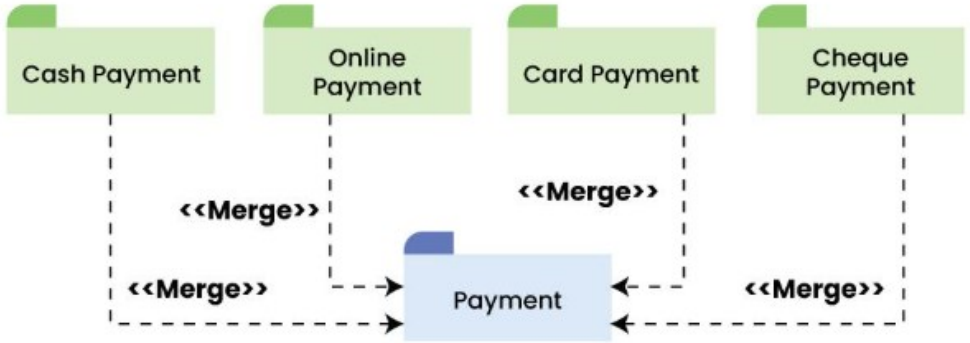


Package Diagrams : Types of Dependencies

- «access»: indicates that one package requires the assistance or services provided by the functions or elements of another package. It implies a runtime or execution-level dependency between the two packages

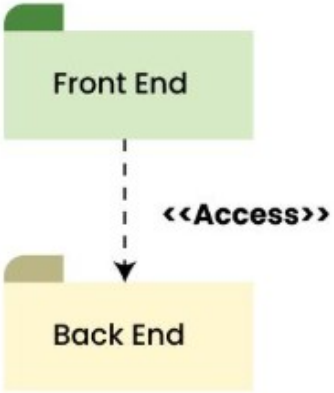
Package Diagrams : Types of Dependencies

- «merge»: is used to represent that the contents of a package can be merged with the contents of another package. This implies that the source and the target package has some elements common in them, so that they can be merged together.

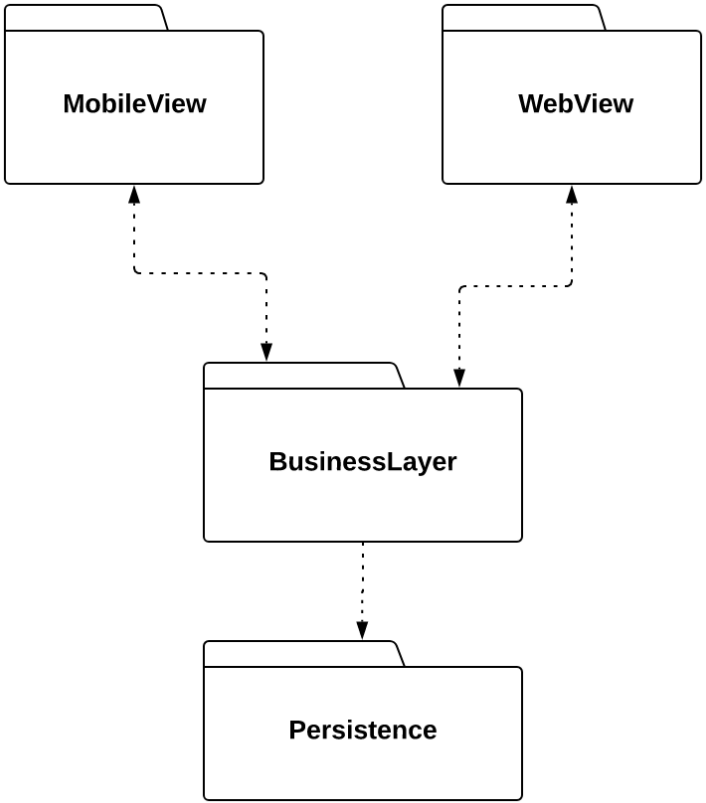


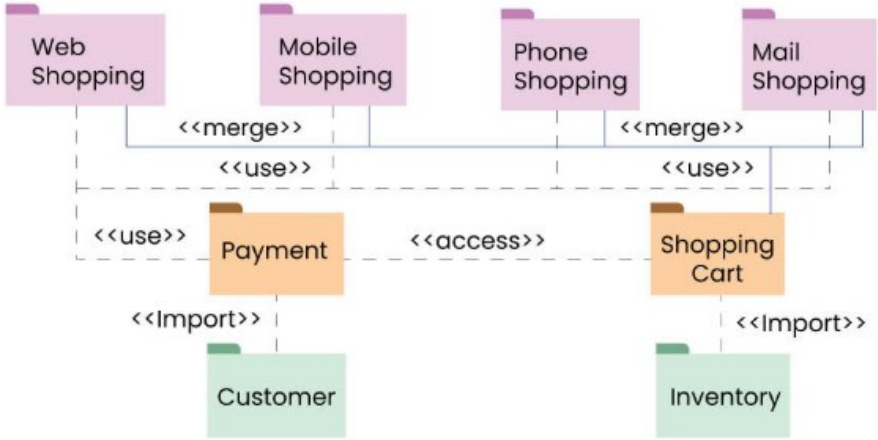
Package Diagrams : Types of Dependencies

- «access»: signifies that there is a access relationship between two or more packages, meaning that one package can access the contents of another package without importing it.

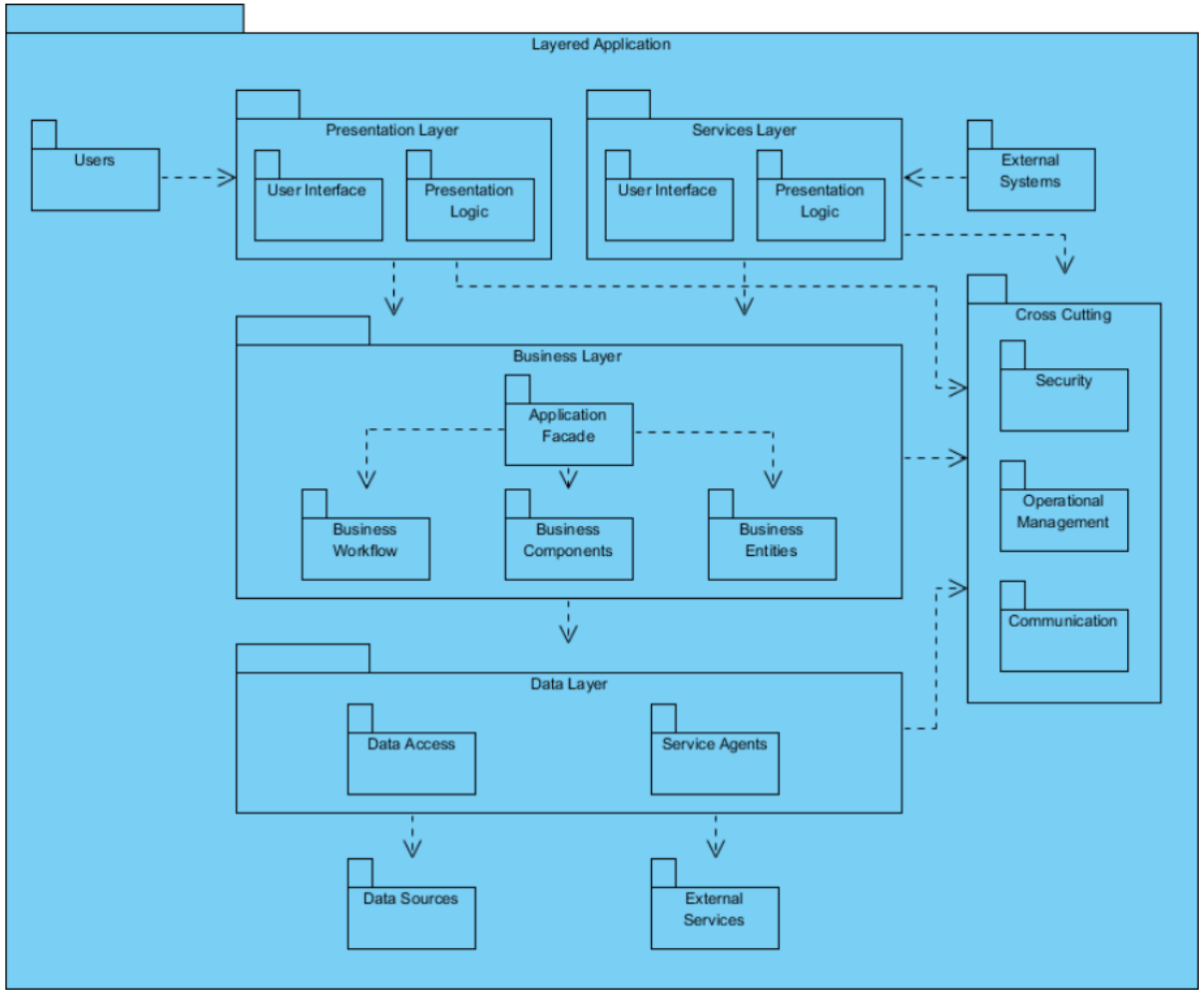


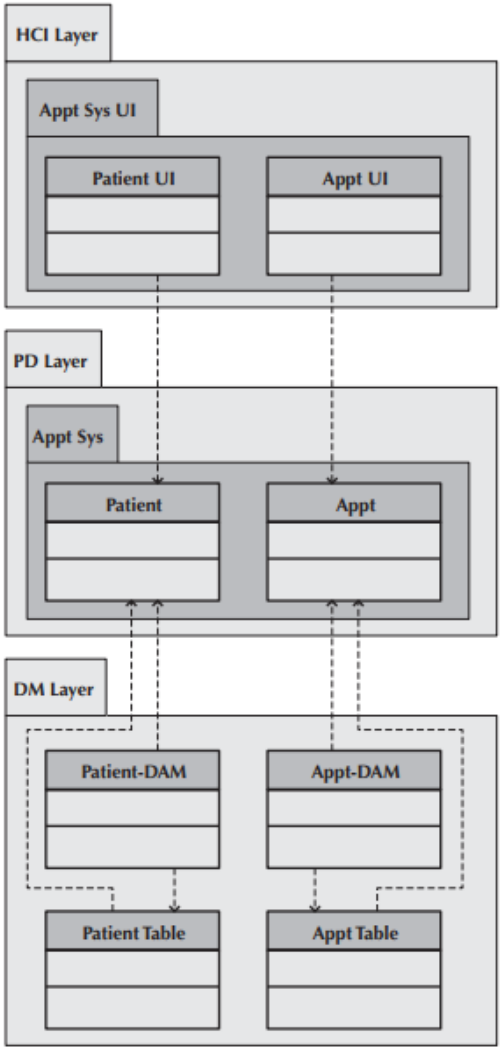
Package Diagrams : examples

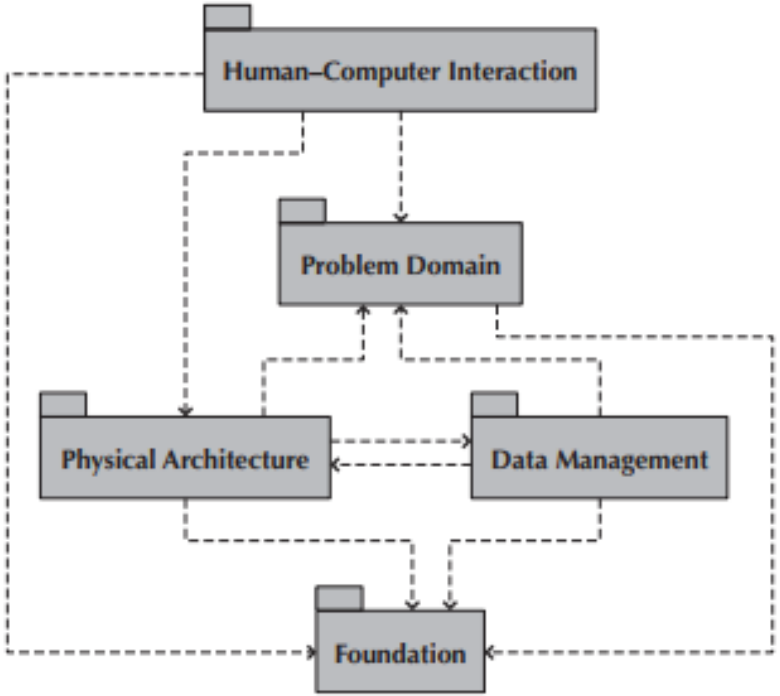




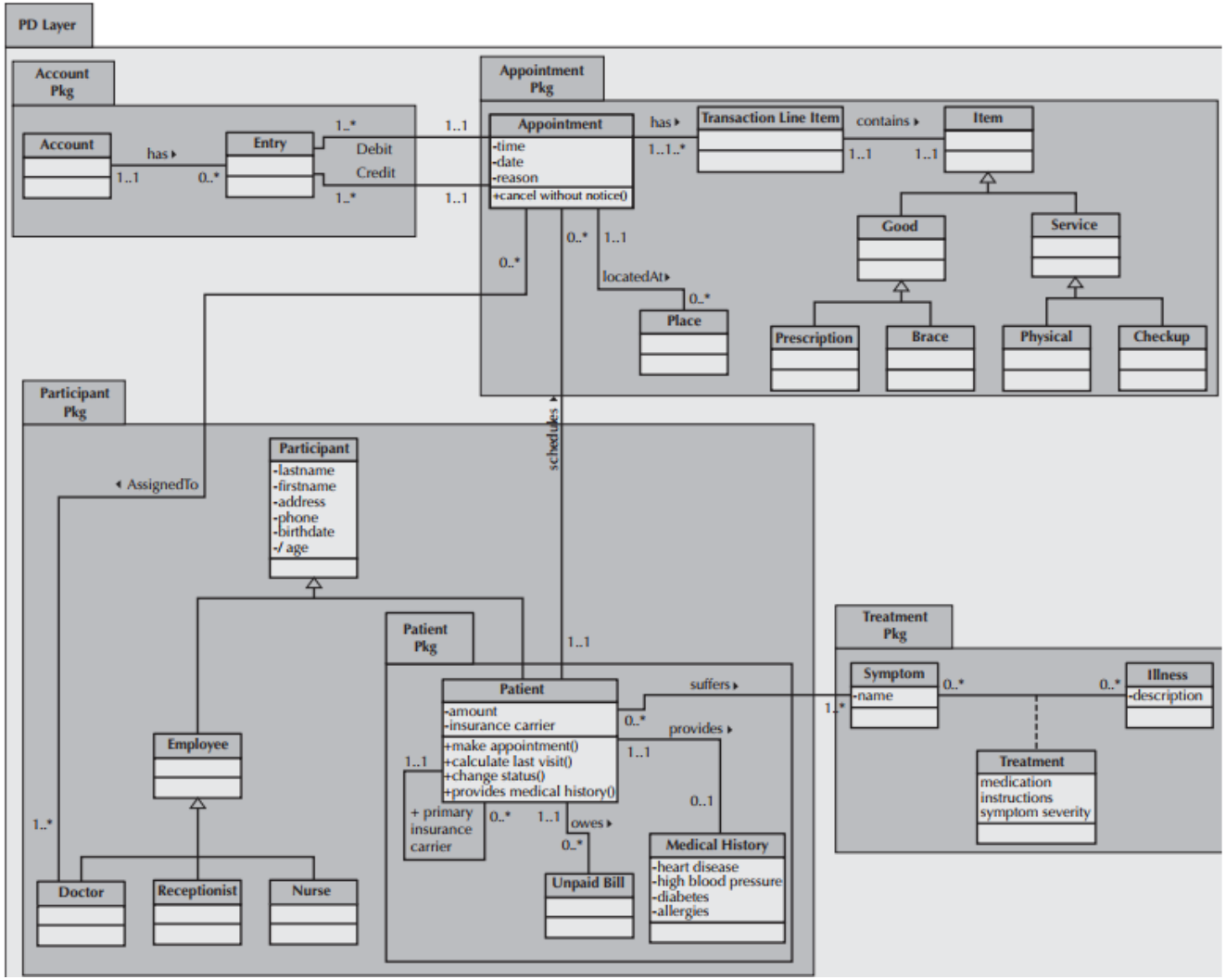
Package Diagrams




HCI Layer
Appt Sys UI
Patient UI
Appt UI
PD Layer
Appt Sys
Patient
Appt
DM Layer
Patient-DAM
Appt-DAM
Patient Table
Appt Table


Human–Computer Interaction
Problem Domain
Physical Architecture
Data Management
Foundation

Package Diagrams : examples



Behavior Modeling

- Verbs in the requirements
- Dynamic parts of UML models: “behavior over time”
- Usually connected to structural elements

Two primary kinds of behavioral elements:

- **Interaction**

  a set of objects exchanging message to accomplish a specific purpose

- **Dynamics/ State Machine**

  specifies a sequences of states an object goes through during its lifetime in response to events.

Behavior Modeling

- Statechart diagram
  - Depicts the flow of control inside an object using **states** and **transitions** (finite state machines)
- Activity diagram
  - Describes the **control flow** among objects by actions organized in workflows

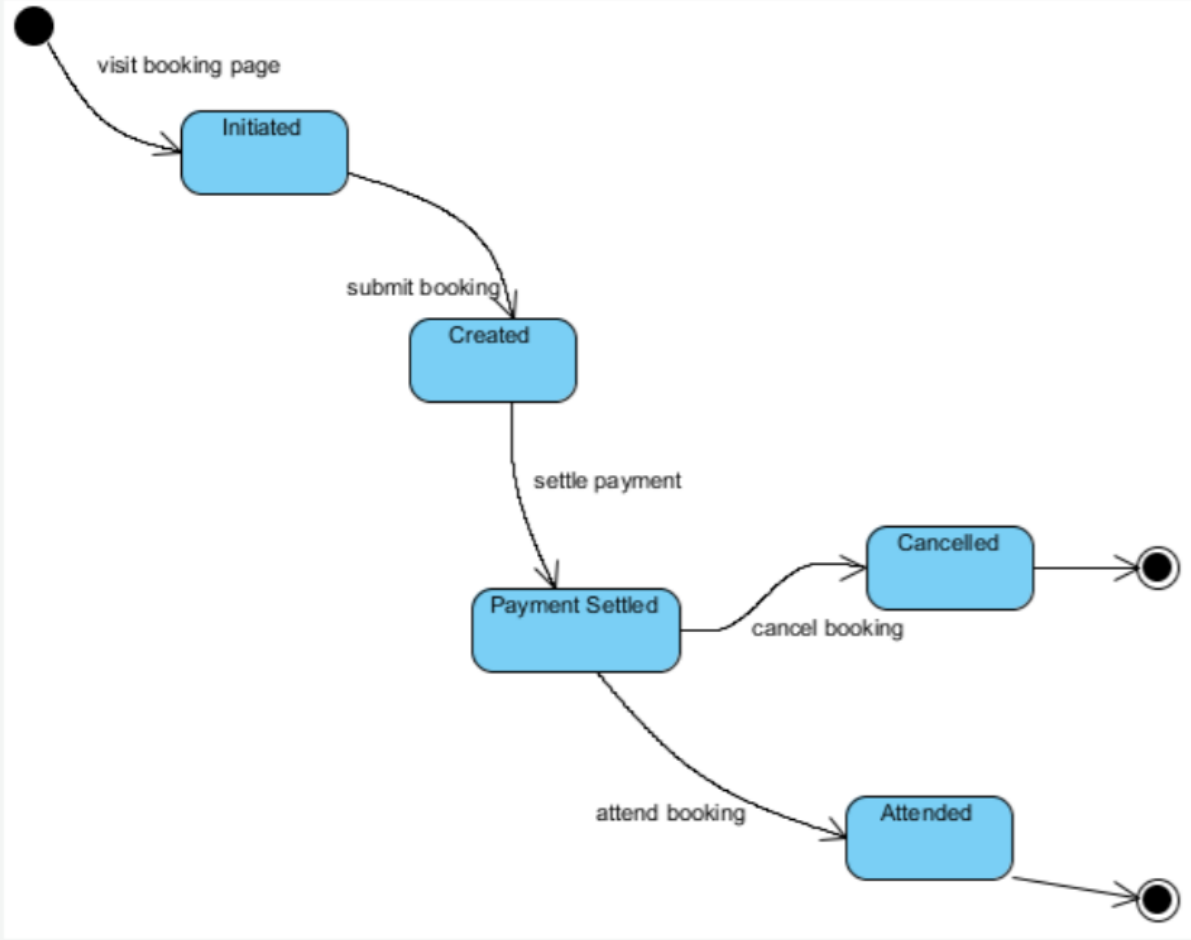
OBJETC'S DYNAMICS

- Sequence diagram
  - Depicts objects' interaction by highlighting the **time ordering** of method invocations
- Communication (collaboration) diagram
  - Depicts the **message flows** among objects

OBJETCS' INTERACTION

State Machine Diagrams (Statecharts)

State diagram: booking object

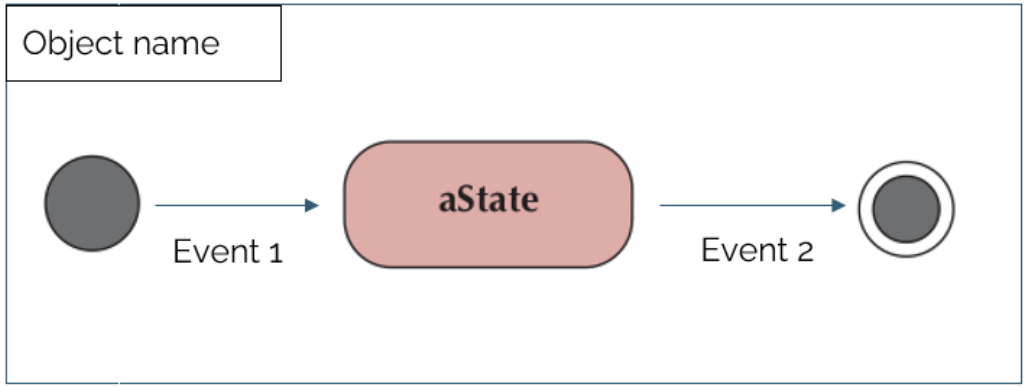


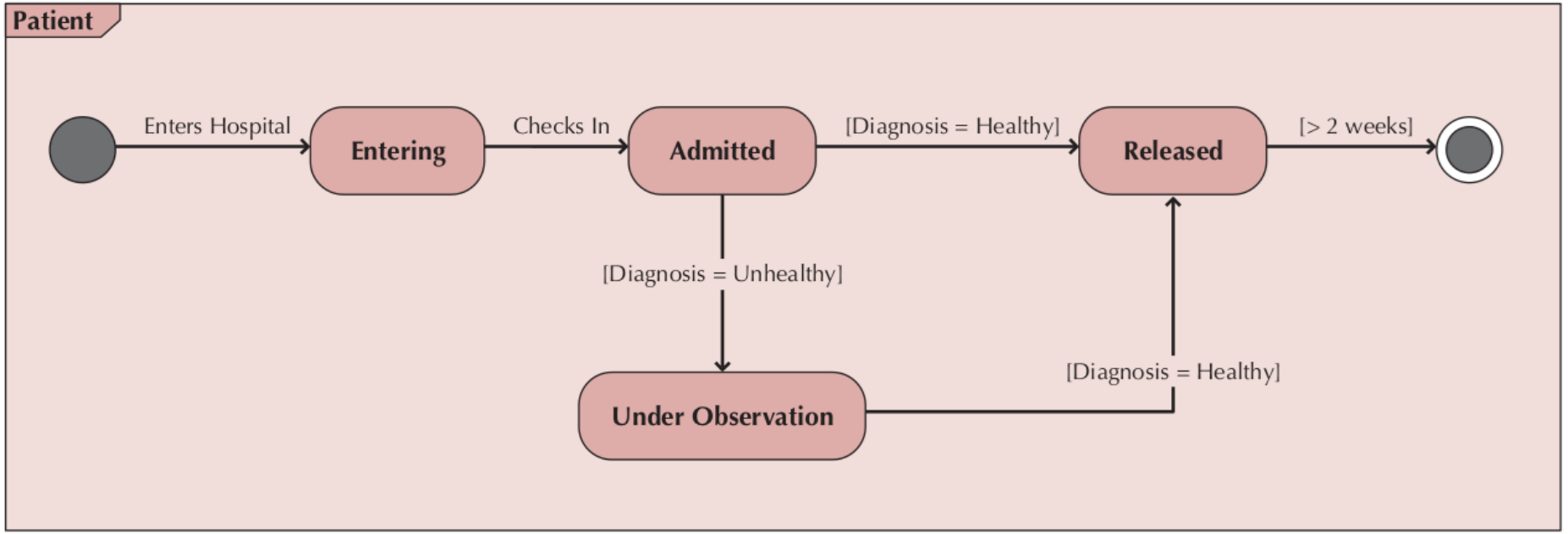
State Diagram

- A state diagram represents the behavior of an object
- Graph: net of states (nodes) and transitions (arrows)
- Graph representing a finite state machine
- Useful for modeling a reactive (event-driven) system

State Diagram : Elements

- Components
  - *Object Box [optional]*
  - *States:*
    - *Initial*
    - *Final/Exit State*
    - *State*
  - *Transitions*
    - *Arrow*
    - *Event Label*
  - *Other Elements : notes,...*




Patient
Enters Hospital
Entering
Checks In
Admitted
[Diagnosis = Healthy]
Released
[> 2 weeks]
[Diagnosis = Unhealthy]
Under Observation
[Diagnosis = Healthy]

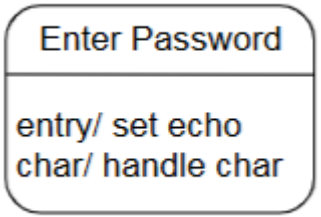
State Diagram : State

- Situation in the life of an object (or system) during which it:
  - Satisfies some condition,
  - Performs some activity, or
  - Waits for some events

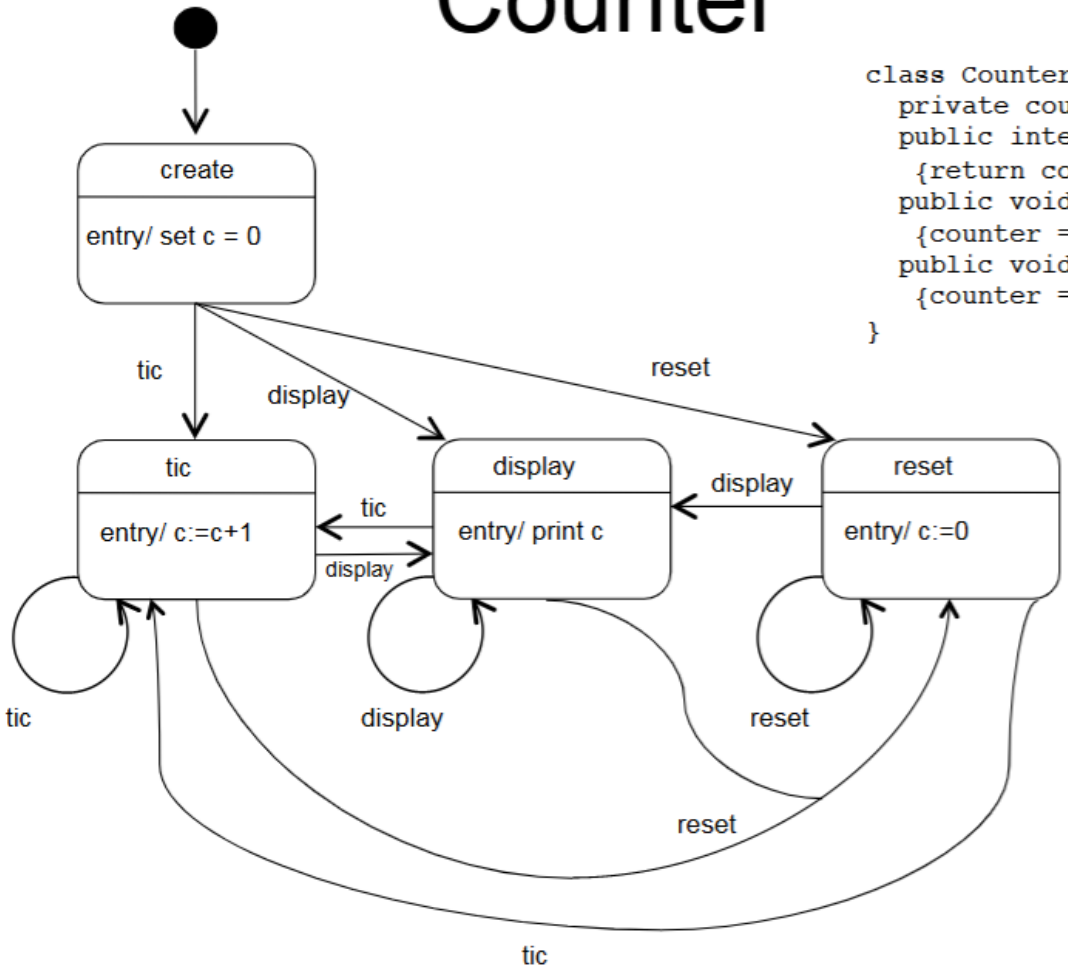
- Set of values of properties that affect the behavior of the object (or system)
  - Determines the response to an event,
  - Thus, different states may produce different responses to the same event

State Diagram : State

- States are rounded rectangles with at least one section
  - Mandatory field: name
- optional: list of internal actions (with optional guards)
  - format: event-name argument-list | [guard condition] / action-expression
  - special actions: 'entry/' and 'exit/' (these cannot have arguments or guards)
- optional: invoking a nested state machine
  - format: do/machine-name
  - 'machine-name' must have initial and final states



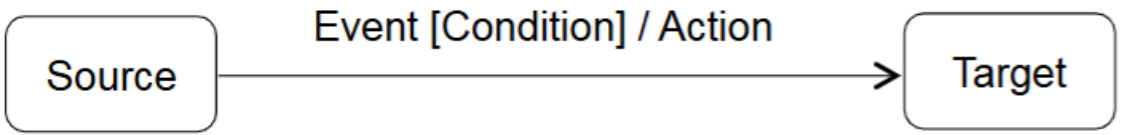
Counter




```
class Counter{
  private counter: integer;
  public integer display()
   {return counter};
  public void tic()
   {counter = counter + 1};
  public void reset()
   {counter = 0};
}
```

State Diagram : Transition

- Relationship between two states indicating that a system (or object) in the first state will:
  – Perform certain actions and
  – Enter the second state when a specified event occurs or a specified condition is satisfied
- A transition consists of:
  – Source and target states
  – Optional event, guard condition, and action

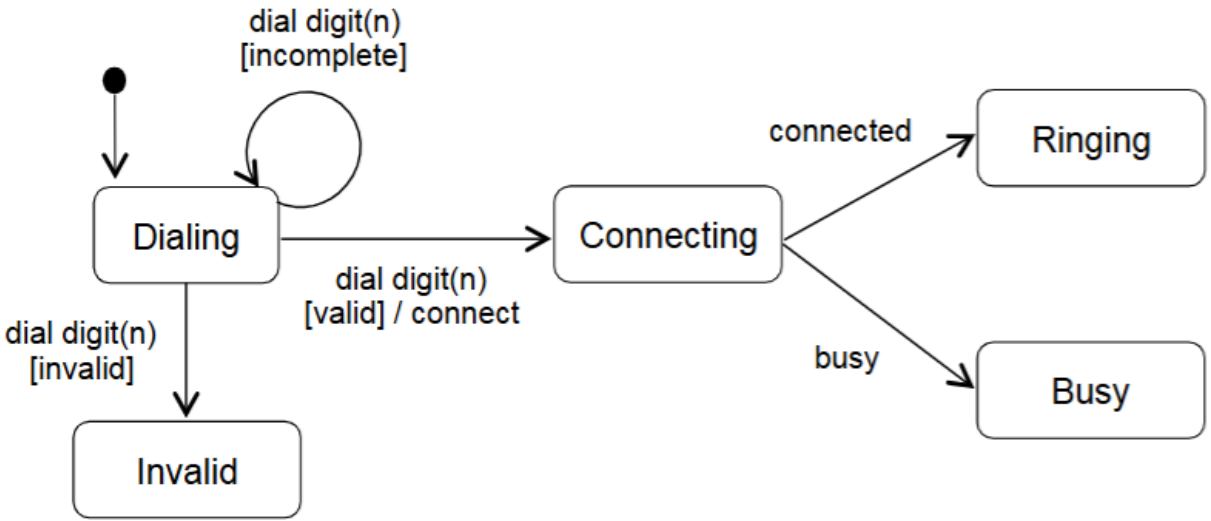


State Diagram : Transition (Event, Action)

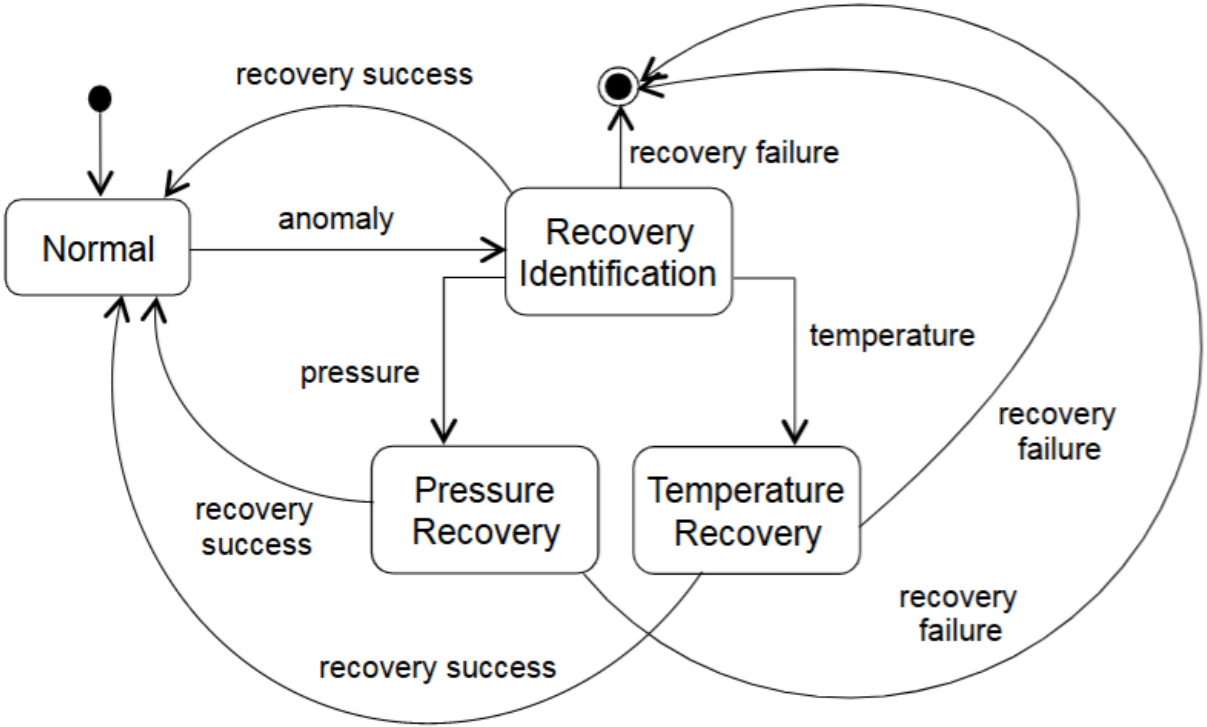
- **Event**
  – An occurrence of a stimulus that can trigger a state transition
  – Instantaneous and no duration

- **Action**
  – An executable atomic computation that results in a change in state of the model or the return of a value

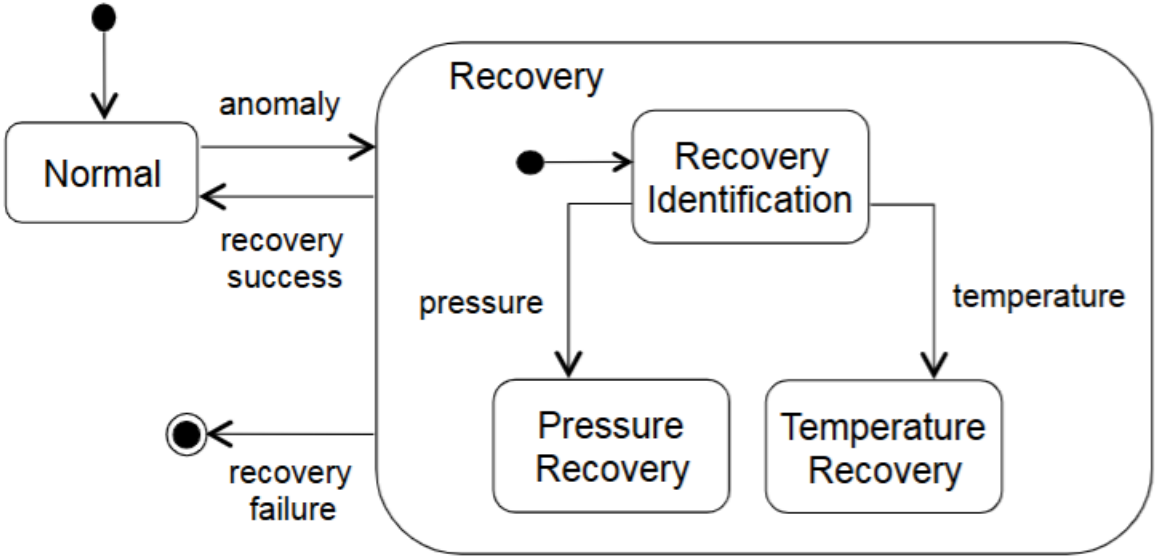
State Diagram : Example



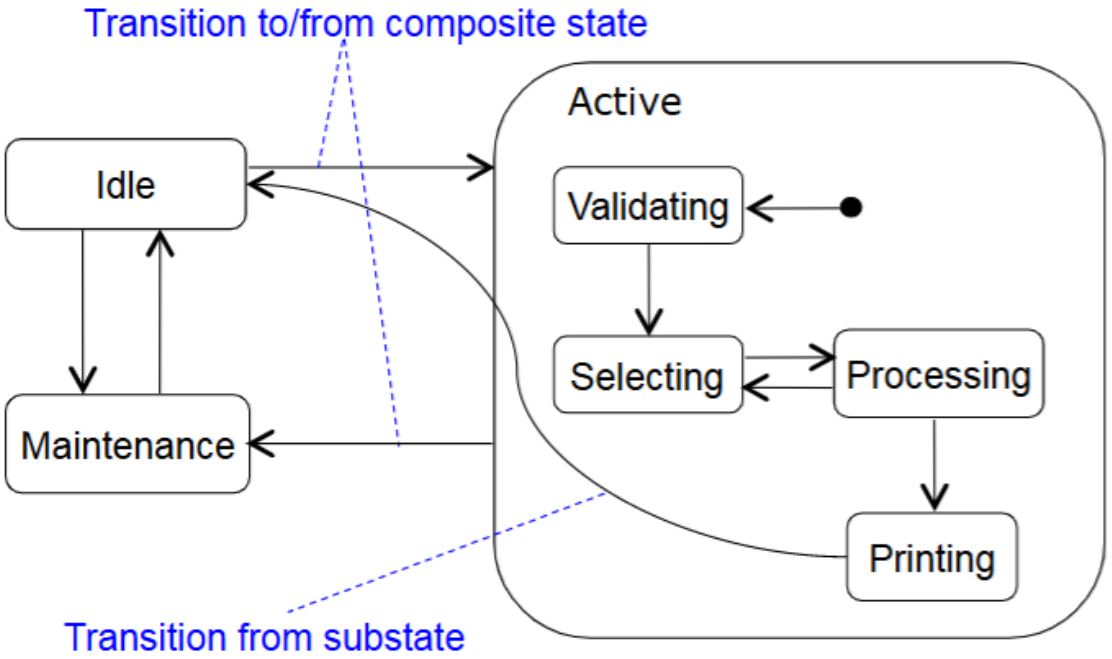
State Diagram : Example



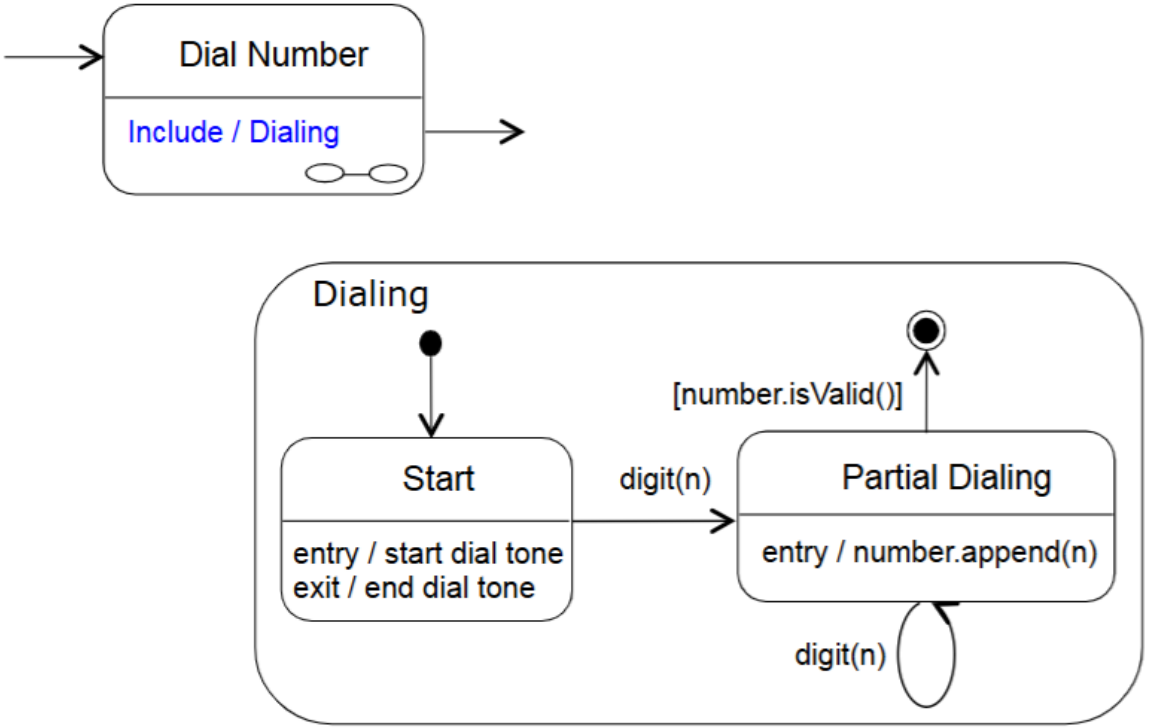
State Diagram : Composite states



State Diagram : Composite states and Transitions



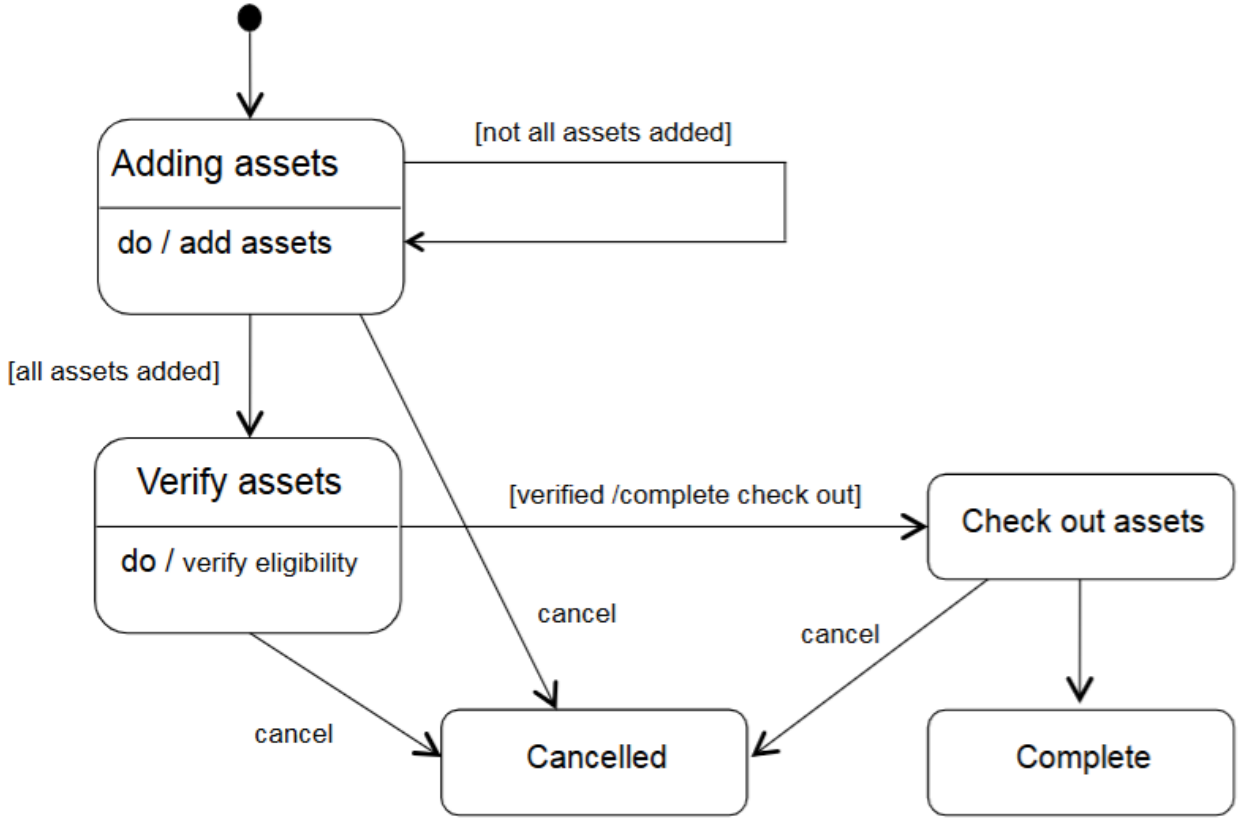
State Diagram : Including composite states



State Diagram : Composite states

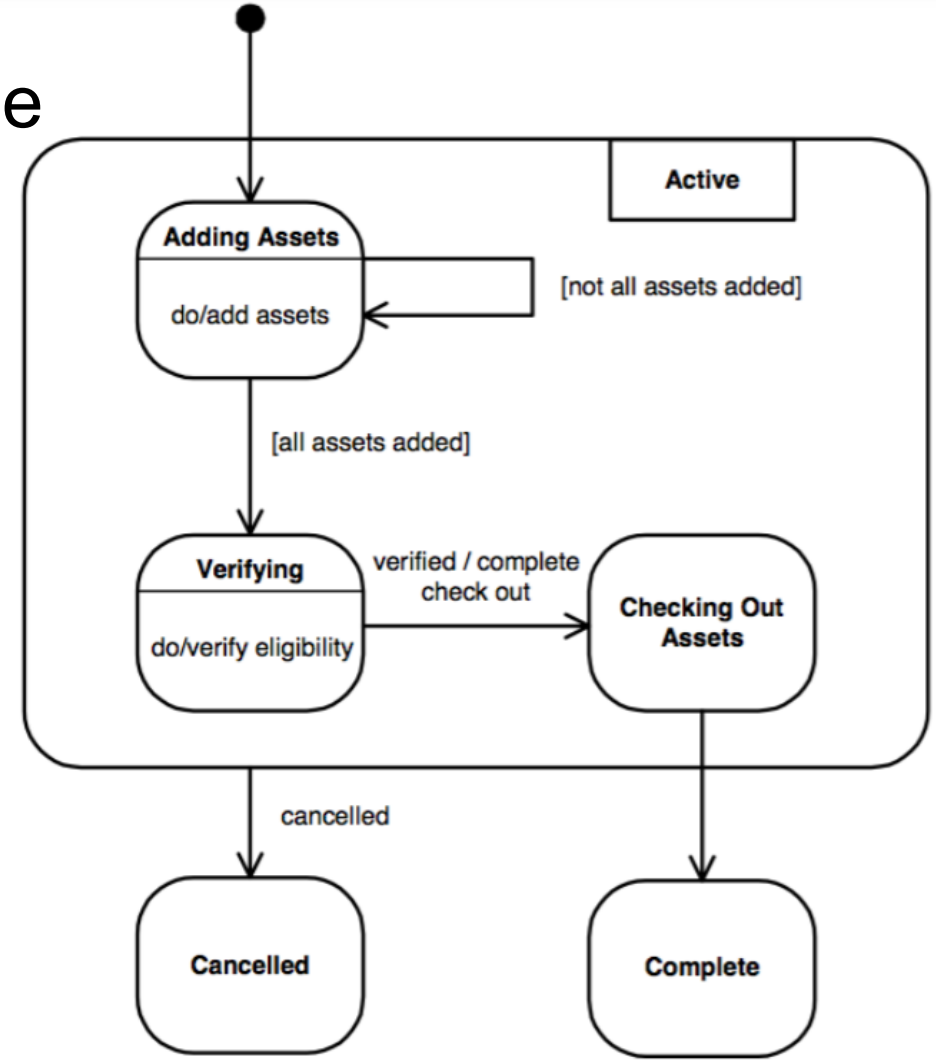
- Used to simplify diagrams
- Inside, it looks like a statechart/state diagram
- It may have composite transitions
- It may have transitions from substates
- It can be sequential or parallel

State Diagram : Example



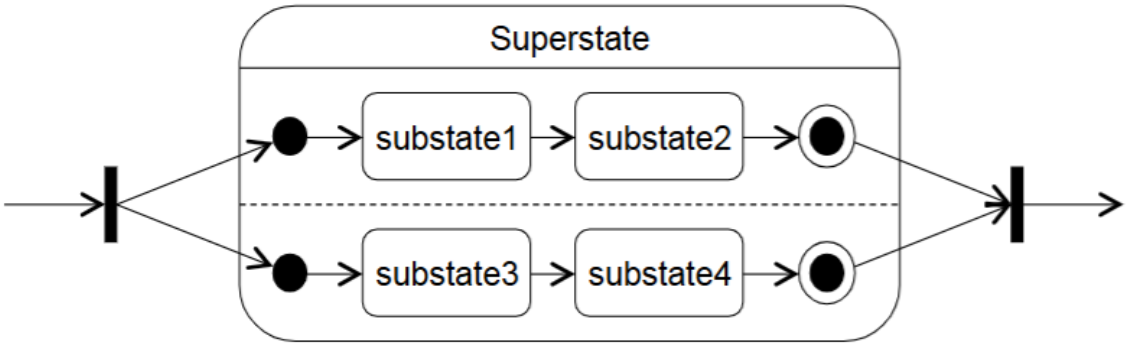
State Diagram : Example

Exploiting a Composite State

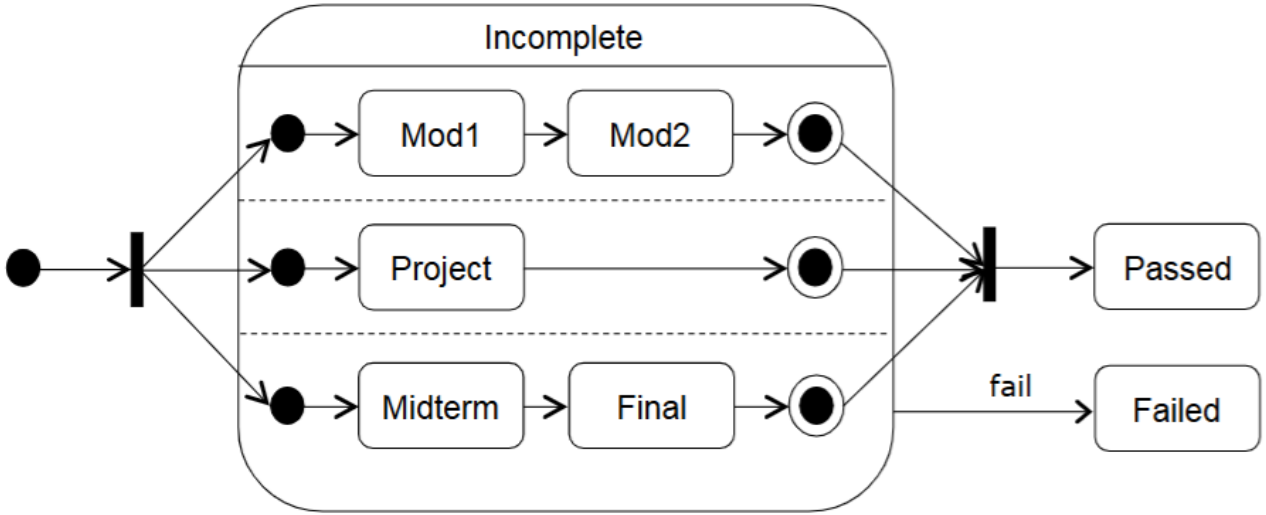


State Diagram: Parallel composition

- Concurrency (multiple threads of control)
- Synchronization



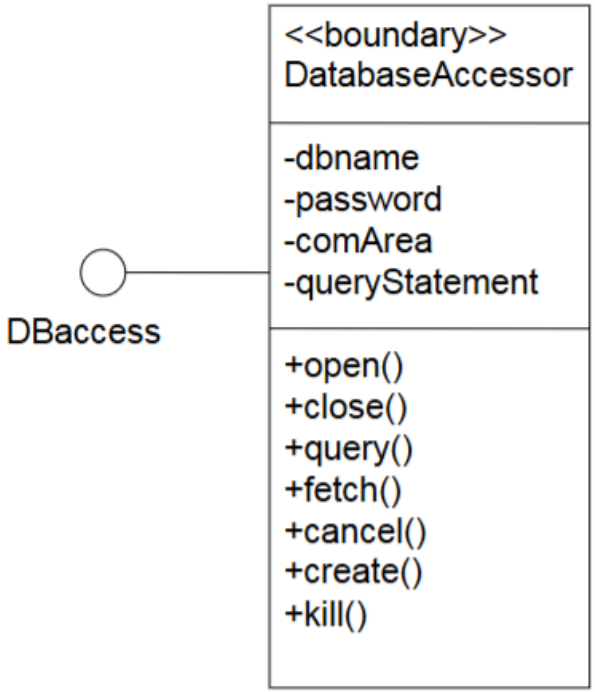
State Diagram: Parallel composition

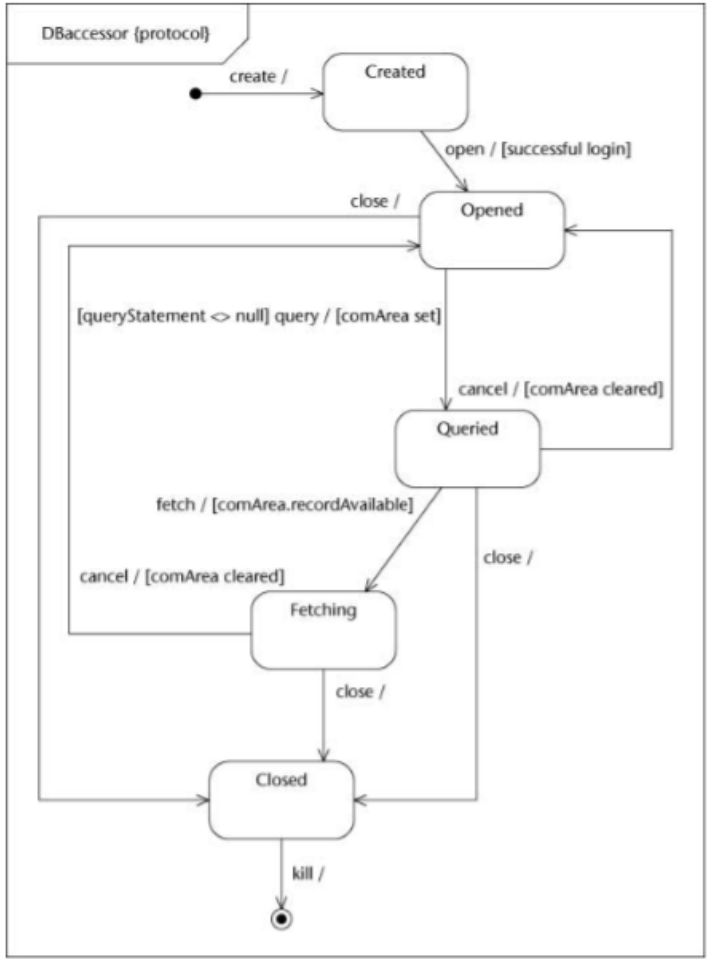


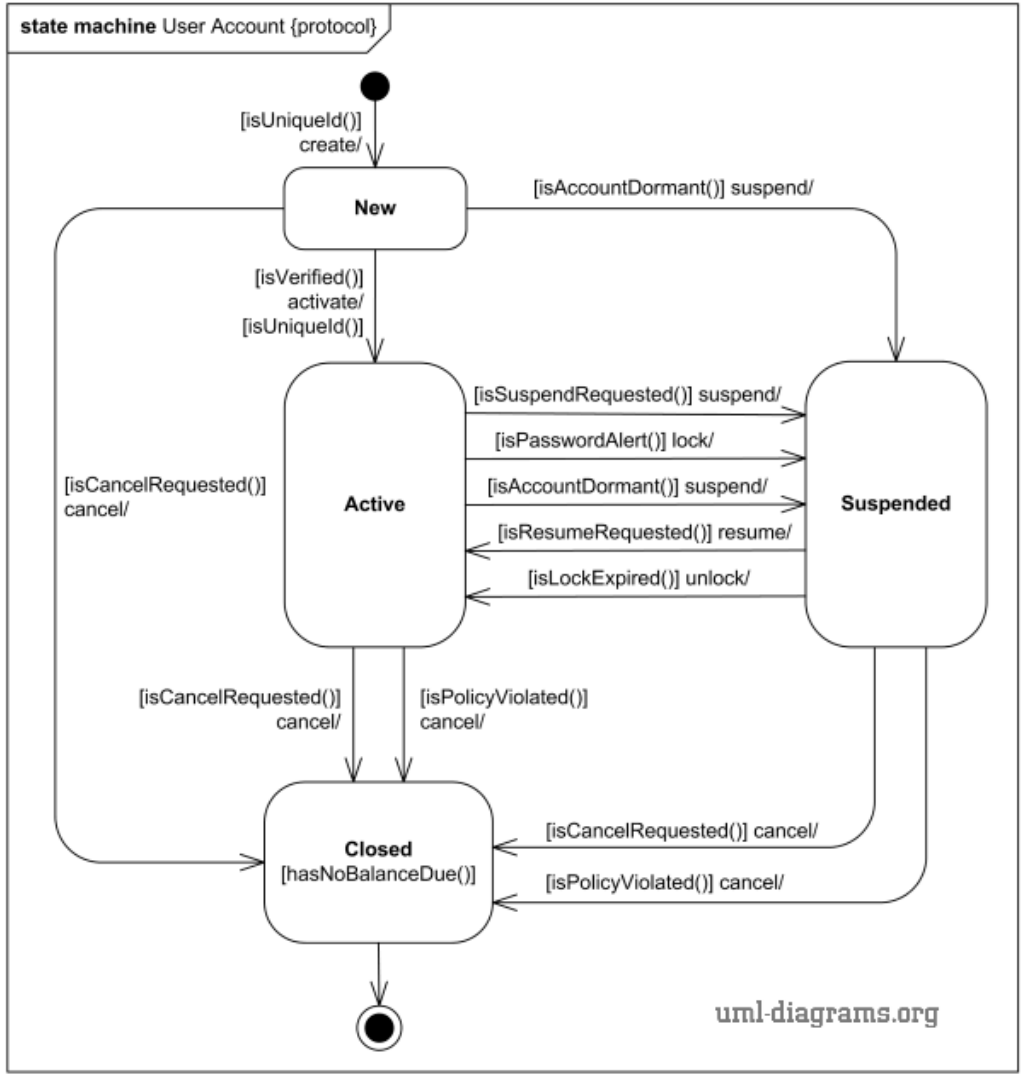
State Diagram: Protocol state machine

- Normally we use a state diagram to show the internal behavior of all objects of a class
- Sometimes, however, we want to show a complex protocol (set of rules governing communication) when using an interface for a class
- For example, when we access a database we need to use operations like open, close and query. But these operations must be called in the right order: we cannot query the database before we open it

Protocol state machine






state machine User Account {protocol}
[isUniqueId()]
create/
New
[isAccountDormant()] suspend/
[isVerified()]
activate/
[isUniqueId()]
[isSuspendRequested()] suspend/
[isPasswordAlert()] lock/
[isAccountDormant()] suspend/
[isResumeRequested()] resume/
[isLockExpired()] unlock/
[isCancelRequested()]
cancel/
Active
Suspended
[isCancelRequested()]
cancel/
[isPolicyViolated()]
cancel/
Closed
[hasNoBalanceDue()]
[isCancelRequested()] cancel/
[isPolicyViolated()] cancel/
uml-diagrams.org

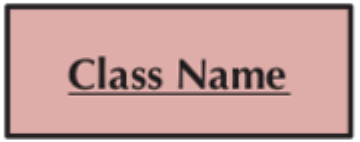
Activity Diagram

Activity Diagram

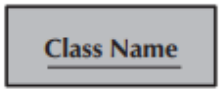
- Activity diagrams represent workflows of actions of several objects (but objects are not shown)
- Actions are composed by sequence, choice, iteration, and concurrency
- AD can be used to describe the activities of the components of a system
- In UML2 they have a different semantics, they can be used to model the behavior of a system, component, use case or even algorithm

Activity Diagram : Elements

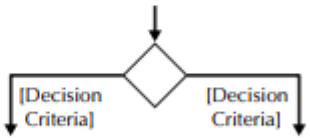
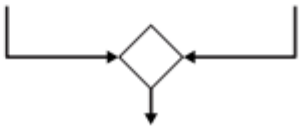
- Components
  - *Activity (set of action) and Actions (non-decomposable)*
  - *Object Node : To represent data/object*
  - *Transitions*
    - *Solid Line : Transition from activity to another activity*
    - *Dashed Line : Transition*
  - *Swimlines : Used to contain activities related to a specific actor*



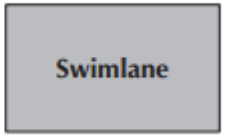
Activity Diagram : Elements

An action: ■ Is a simple, nondecomposable piece of behavior. ■ Is labeled by its name.	
An activity: ■ Is used to represent a set of actions. ■ Is labeled by its name.	
An object node: ■ Is used to represent an object that is connected to a set of object flows. ■ Is labeled by its class name.	
A control flow: ■ Shows the sequence of execution.	
An object flow: ■ Shows the flow of an object from one activity (or action) to another activity (or action).	

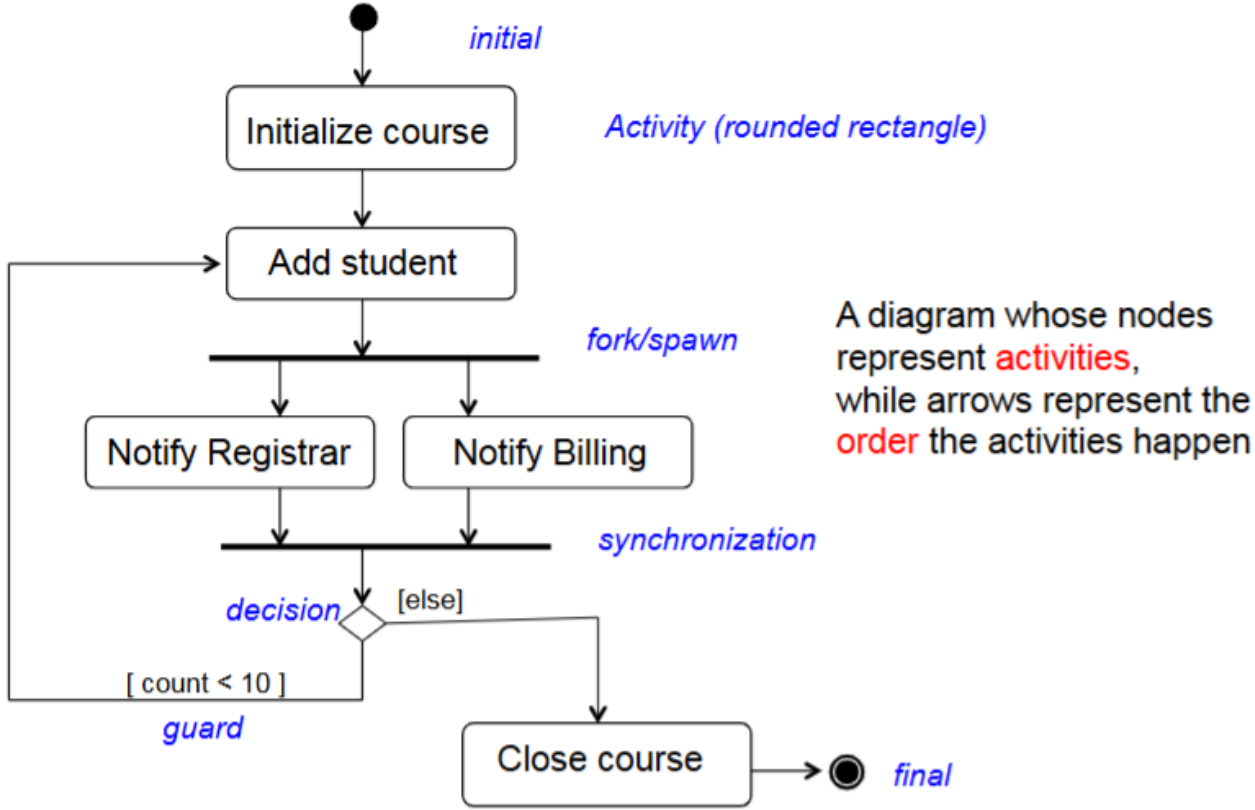
Activity Diagram : Elements

An initial node: ■ Portrays the beginning of a set of actions or activities.	
A final-activity node: ■ Is used to stop all control flows and object flows in an activity (or action).	
A final-flow node: ■ Is used to stop a specific control flow or object flow.	
A decision node: ■ Is used to represent a test condition to ensure that the control flow or object flow only goes down one path. ■ Is labeled with the decision criteria to continue down the specific path.	
A merge node: ■ Is used to bring back together different decision paths that were created using a decision node.	

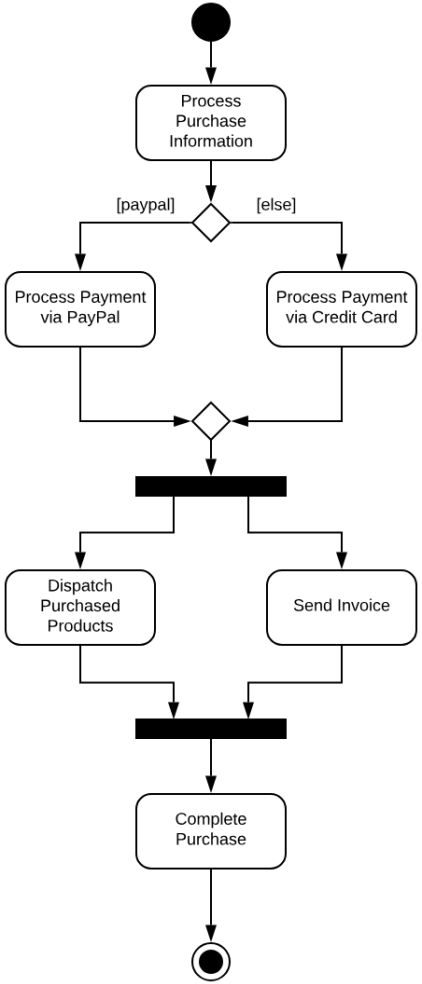
Activity Diagram : Elements

A Fork node: Is used to split behavior into a set of parallel or concurrent flows of activities (or actions).	
A Join node: Is used to bring back together a set of parallel or concurrent flows of activities (or actions).	
A Swimlane: Is used to break up an activity diagram into rows and columns to assign the individual activities (or actions) to the individuals or objects that are responsible for executing the activity (or action). Is labeled with the name of the individual or object responsible.	

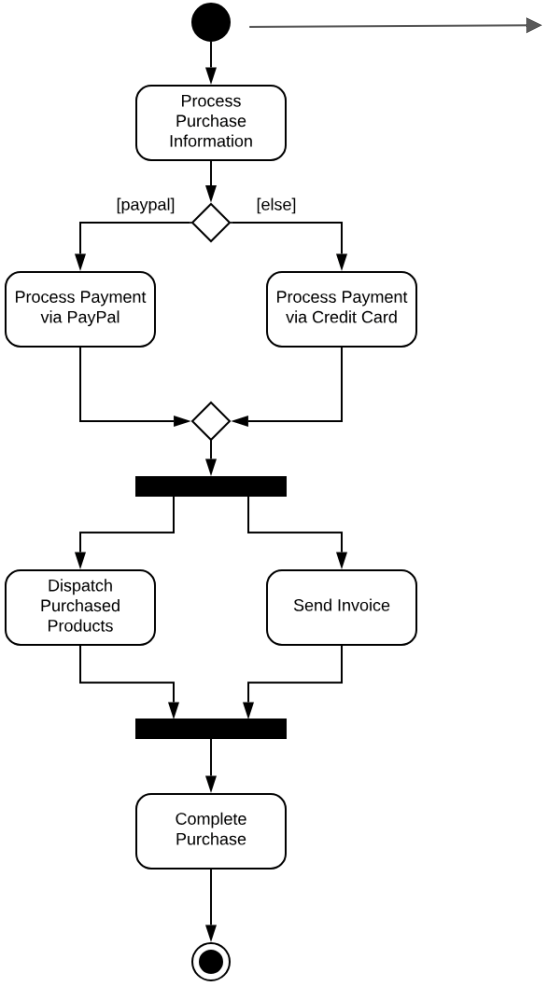
Activity Diagram : Example

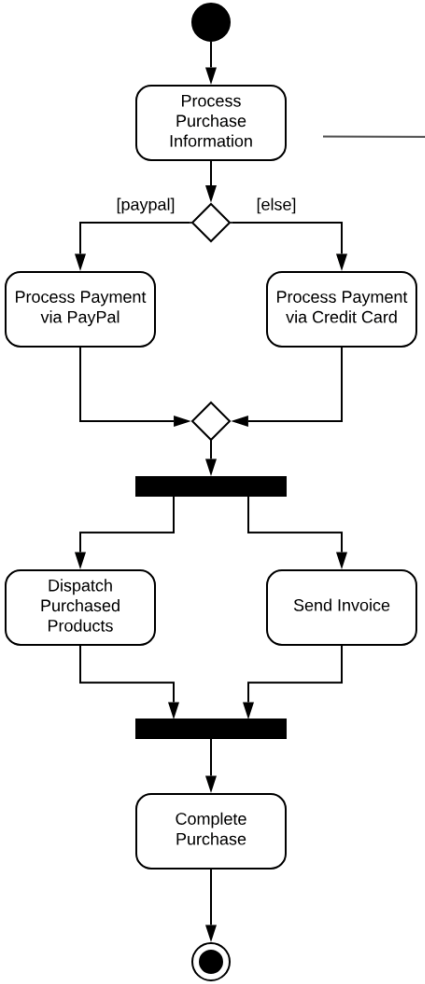


A diagram whose nodes represent activities, while arrows represent the order the activities happen

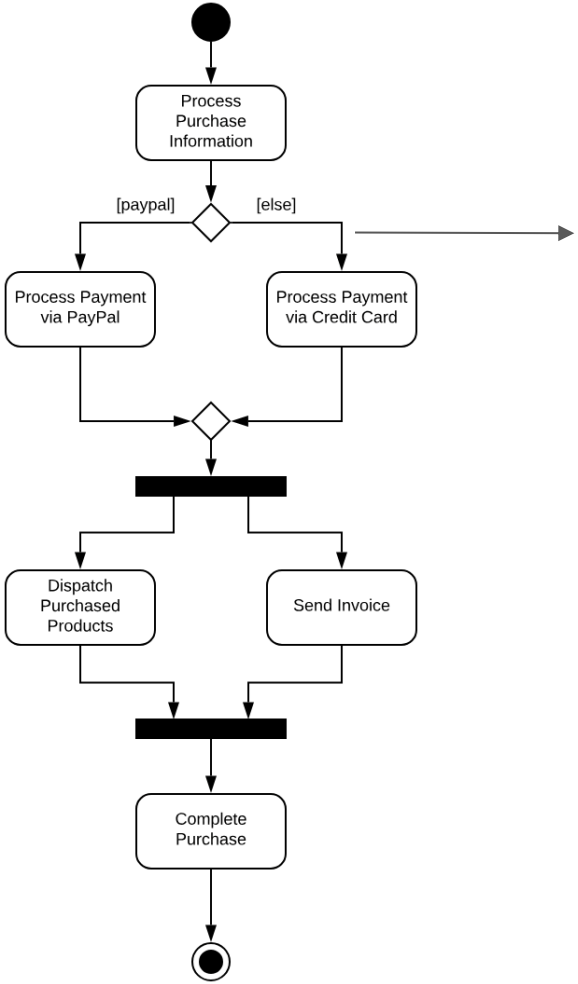


A control token moves through the nodes of the diagram

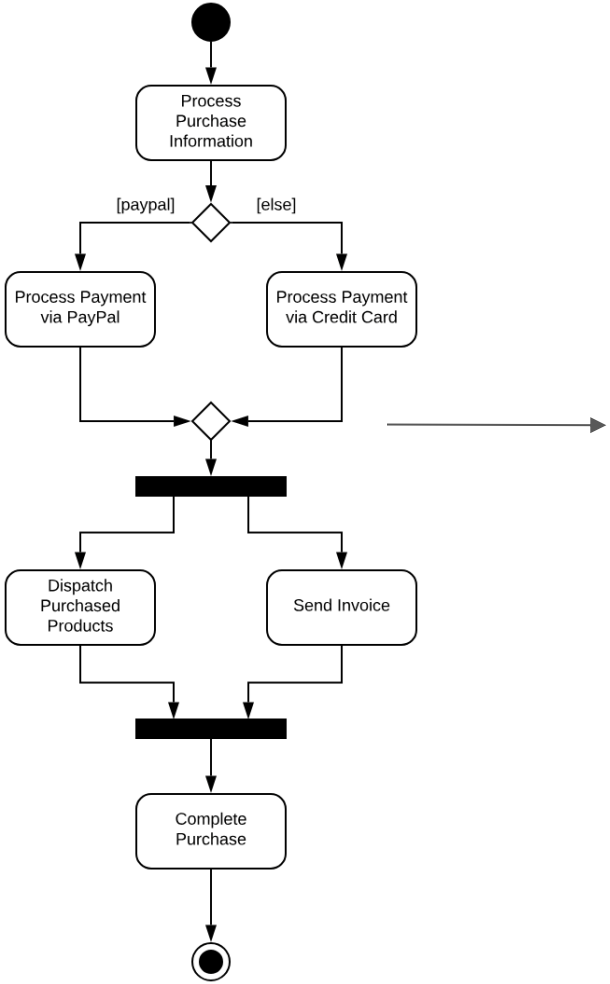

Initial node (creates a token)
Process Purchase Information
[paypal]
[else]
Process Payment via PayPal
Process Payment via Credit Card
Dispatch Purchased Products
Send Invoice
Complete Purchase



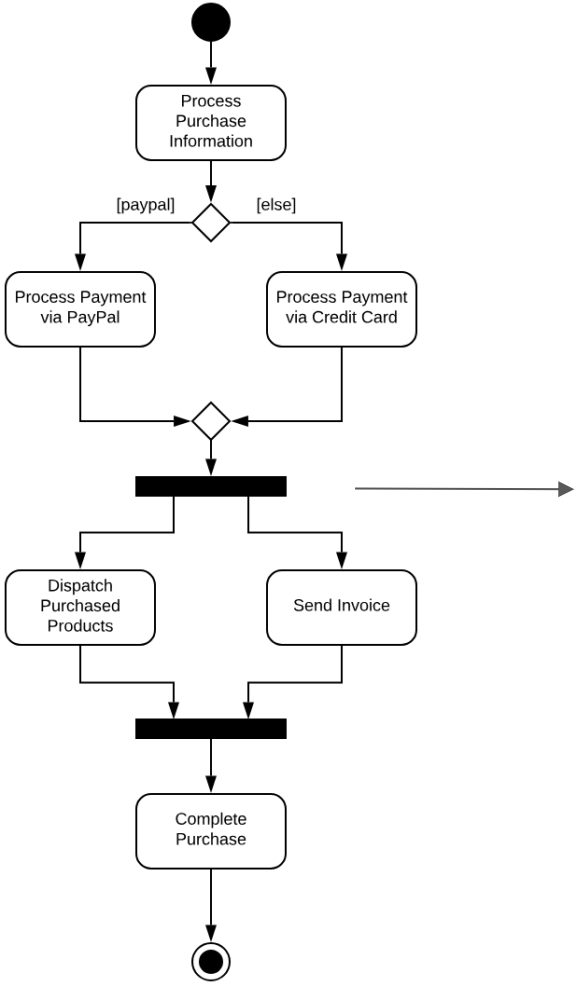
Action (passes a token from input to output flow)



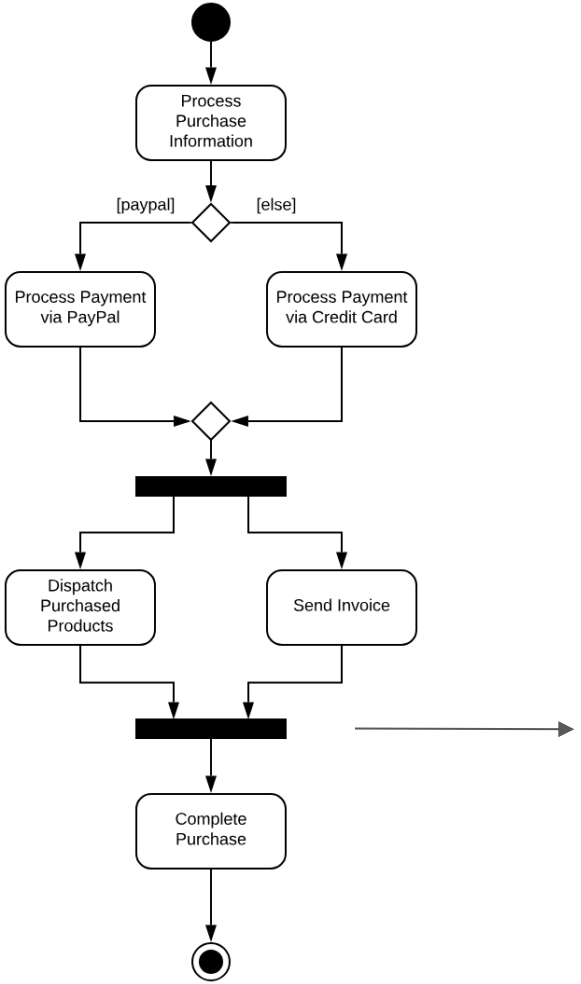
Decision (determines which output flow receives a token)



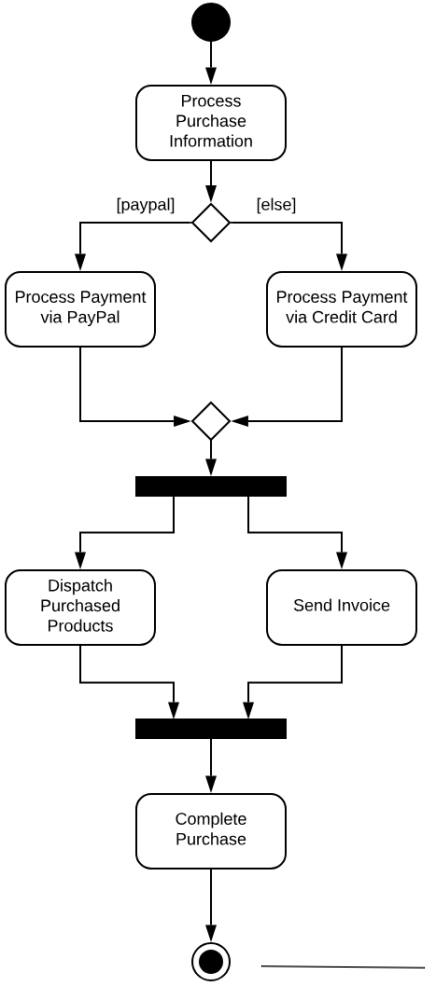
Merge (when a token arrives at one of the inputs, it is passed to the output)



Fork (creates copies of a token and distributes them to all output flows)

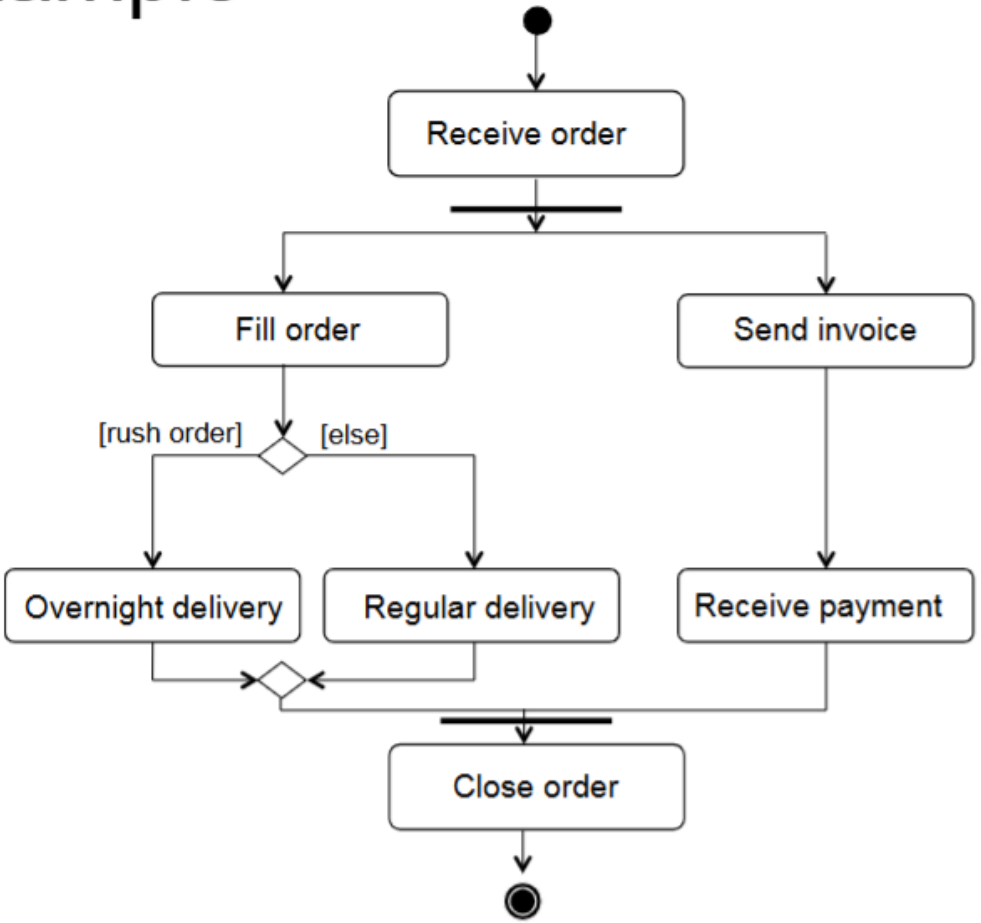


Join (waits for tokens from all input flows before passing one token to the output)

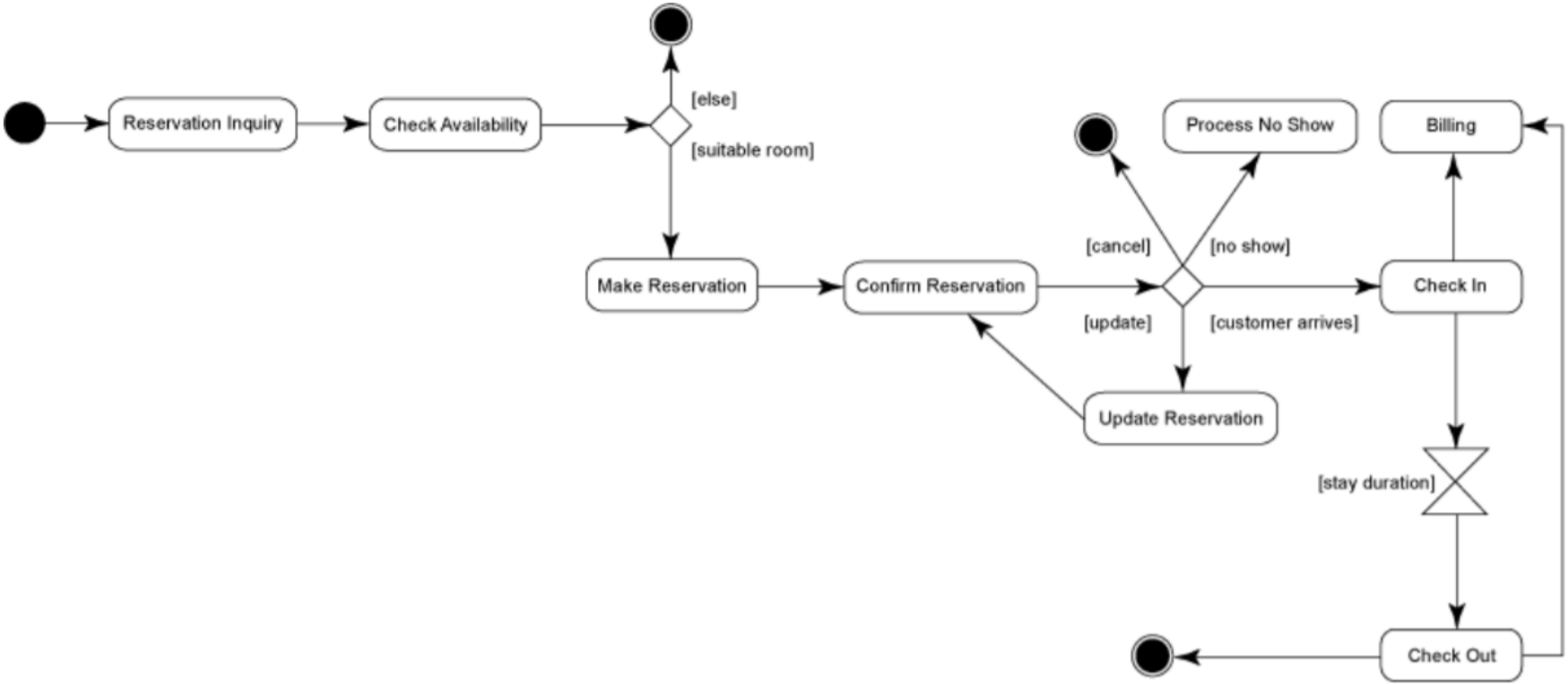


Final node (terminates execution)

Activity Diagram : Example



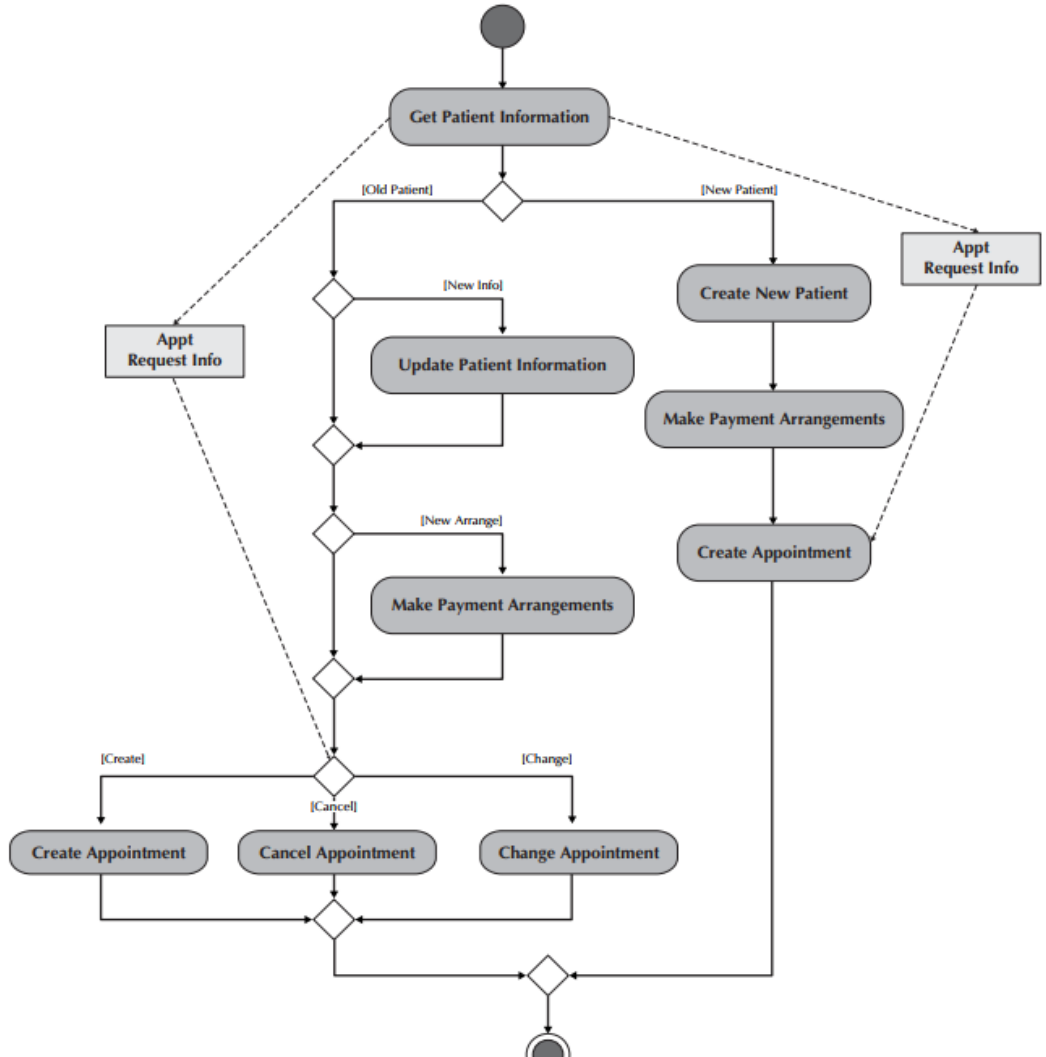
Activity Diagram : Hotel Reservation example



Activity Diagram

Swimlanes
in an
activity diagram
showing a
workflow
with several roles

Activity Diagram for the Manage Appointments Use Case



Activity Diagram : How to create an AD

1. Identify the activities (steps) of a process
2. Identify who/what performs activities (process steps)
3. Identify decision points (if-then)
4. Determine if a step is in loop
5. Determine if a step is parallel with some other
6. Identify the order of activities, decision points

Example

Activity Diagram :

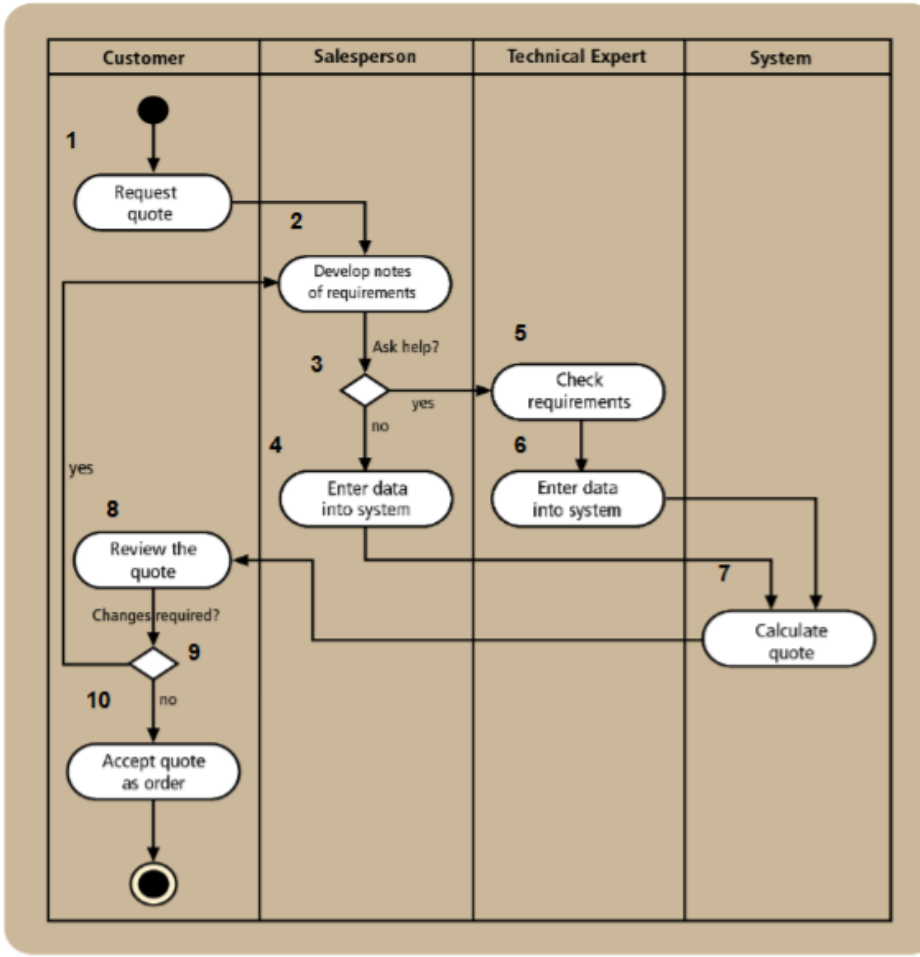
Step ID	Process Step or Decision	Who/What Performs	Parallel Activity	Loop	Preceding Step
1	Request quote	Customer	No	No	-
2	Develop requirement notes	Salesperson	No	Yes	1
3	Decision: Help?	Salesperson	-	Yes	2
4	Salesperson enters data	Salesperson	No	Yes	3
5	Check requirements	Technical Expert	No	Yes	3
6	Tech. expert enters data	Technical Expert	No	Yes	5
7	Calculate quote	System	No	Yes	4, 6
8	Review quote	Customer	No	Yes	7
9	Decision: Changes?	Customer	No	Yes	8
10	Accept quote as order	Customer	No	No	9

Activity Diagram : How to create an AD

7. Draw the swimlanes
8. Draw the start point of the process in the swimline of the first activity (step)
9. Draw the oval of the first activity (step)
10. Draw an arrow to the location of the second step
11. Draw subsequent steps, while inserting decision points and synchronization/loop bars where appropriate
12. Draw the end point after the last step.

Activity Diagram :

Example



Modeling Interactions

- Sequence diagram
  - Depicts objects' interaction by highlighting the **time ordering** of method invocations
- Communication (collaboration) diagram
  - Depicts the **message flows** among objects

INTERACTION

Sequence Diagrams

Sequence Diagrams

- Behavioral (dynamic) diagrams that model:
  - Some of the system's objects
  - The methods these objects execute

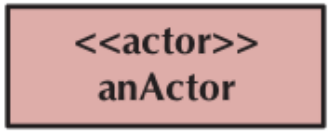
Sequence Diagrams : Main Entities

- **Participant**: an object that acts in the sequence diagram
- **Message**: communication between participant objects
- The **axes** in a sequence diagram:
  - horizontal: which object/participant is acting
  - vertical: time (down -> forward in time)

Sequence Diagrams : Main Entities

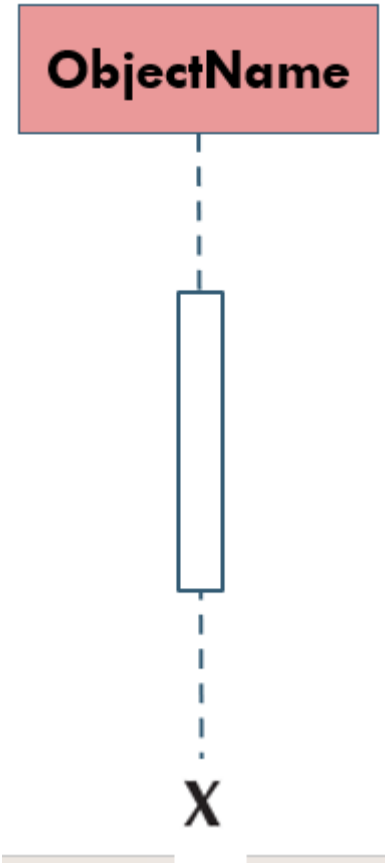
- Participants : Placed at the top of the diagram
  - **Actor** : is a person or an external system using or being by used by the system.
    Depicted as the stick figure for human, if nonhuman is involved, use the rectangle notation
  - **Object** : it participates in the sequence by sending/receiving messages

anActor



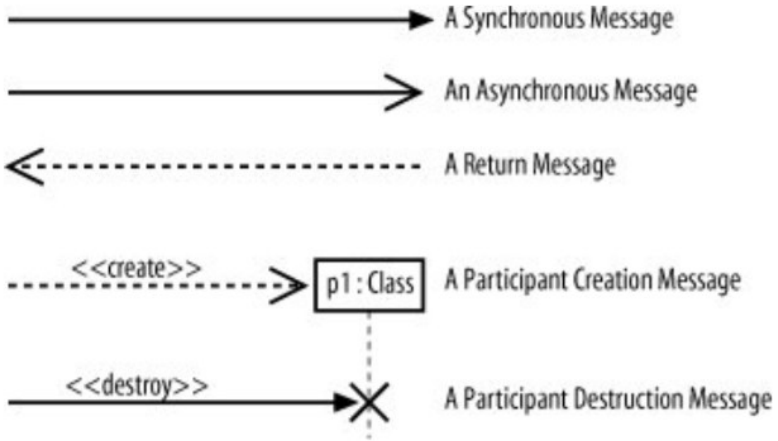
Sequence Diagrams : Main Entities

- Lifespan :
  - Dashed Line : Denotes the life of an object during a sequence
  - Execution Box : Denotes when an object is executing some business logic whilst sending or receiving messages
  - Object Destruction : An X is placed at the end of an object’s lifeline to show that it is going out of existence.

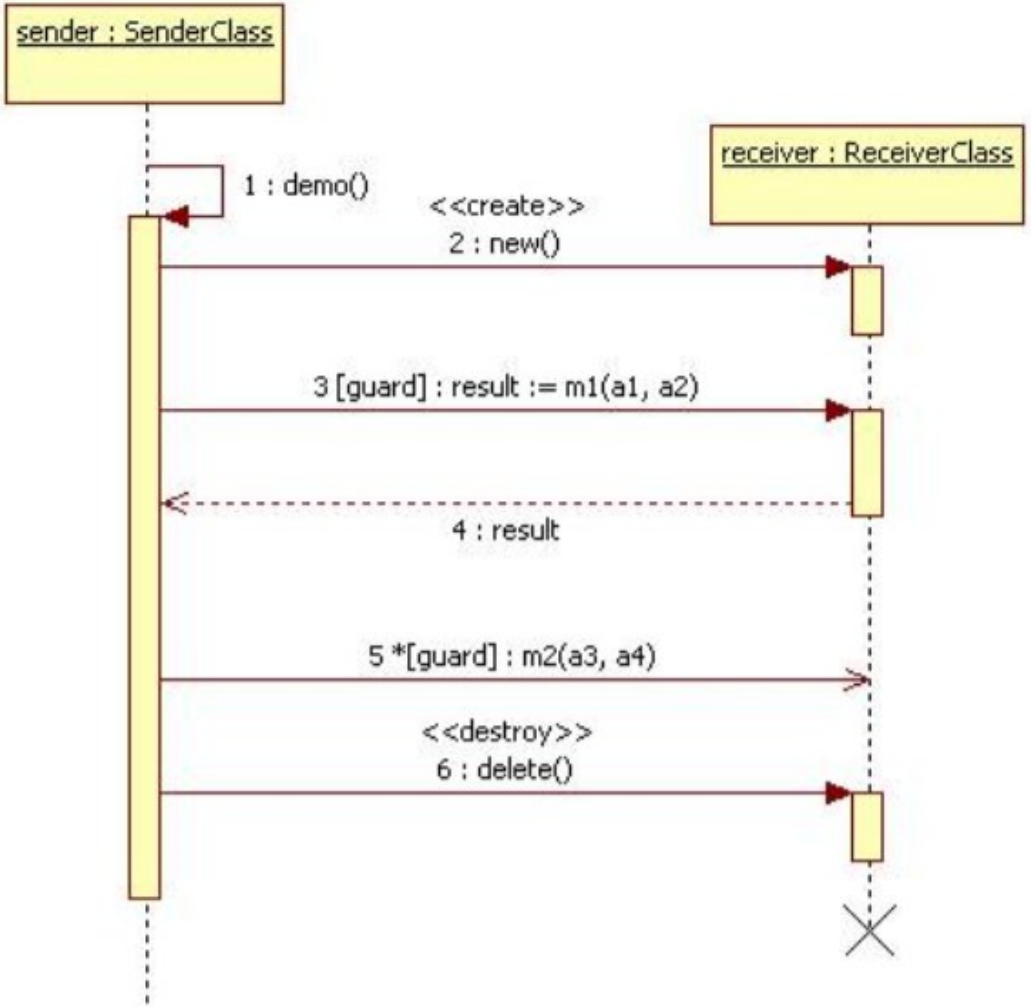


Sequence Diagrams : Main Entities

- **Synchronous Messages or Methods:**
  - *Are represented as a solid line arrow **(filled head)** between the two entities.*
  - *A label for the method is written at the top of the arrow.*
  - *The sender **needs to wait** for the receiver until it completes the execution*
- **Asynchronous Messages :**
  - *The caller does not wait for the receiver to process the message. Represented as a solid line arrow **(Non-filled head)** .*
- **Return Messages :**
  - *Dashed arrow indicates the return message*
- **Reflexive Message :**
  - *When an object calls itself*



Sequence Diagrams : example of messages



Sequence Diagrams : Main Entities

- Frames
  - Can be used for more advanced control including Loops or decision

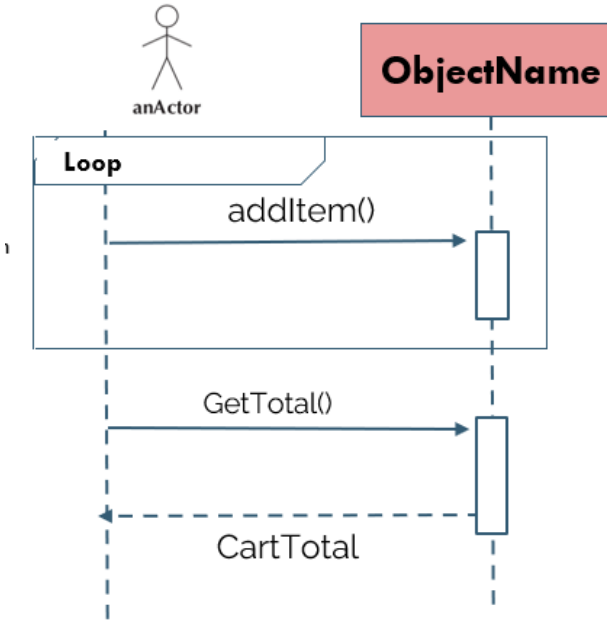
frame: box around part of a sequence diagram to indicate selection or loop

– if : (opt) [condition]!

– if-else: (alt)[condition] else [condition]

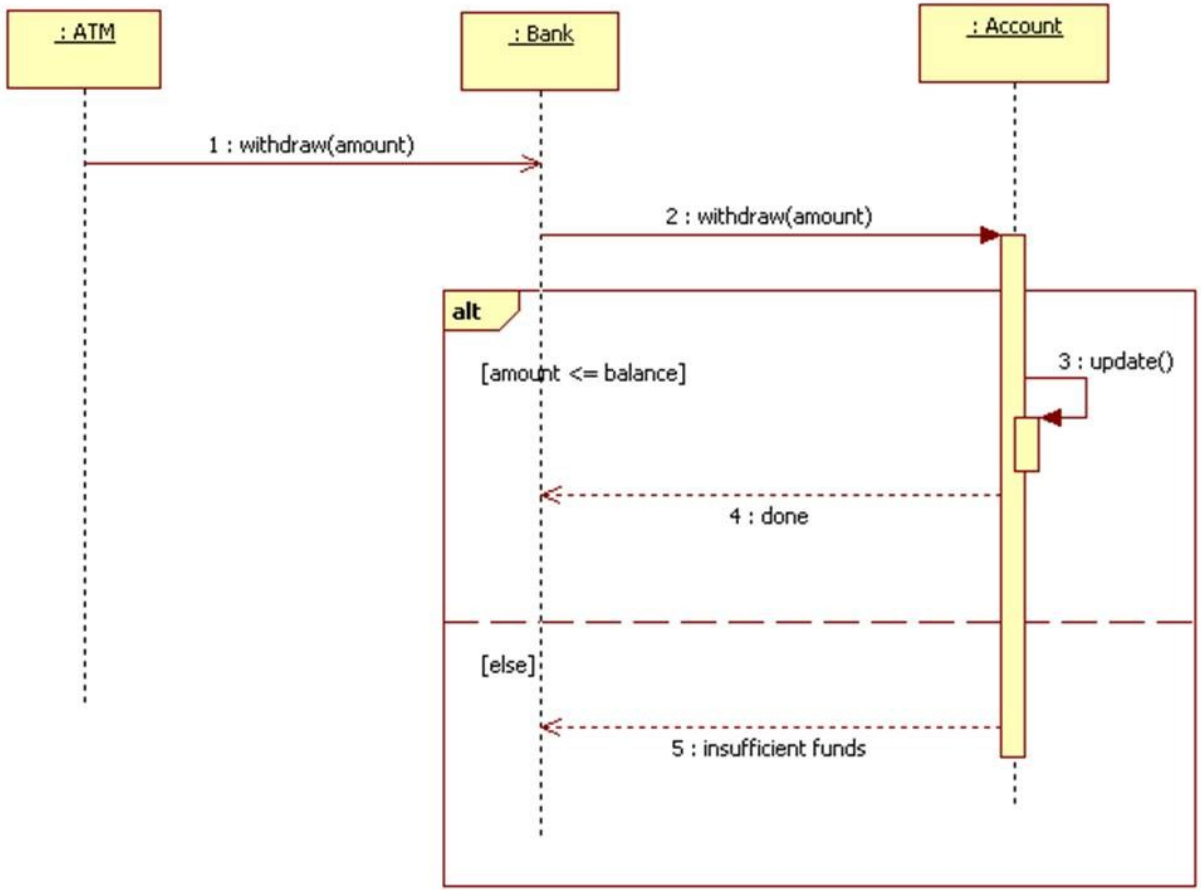
– loop: (loop)[condition to loop over]!

Examples: see the next three slides

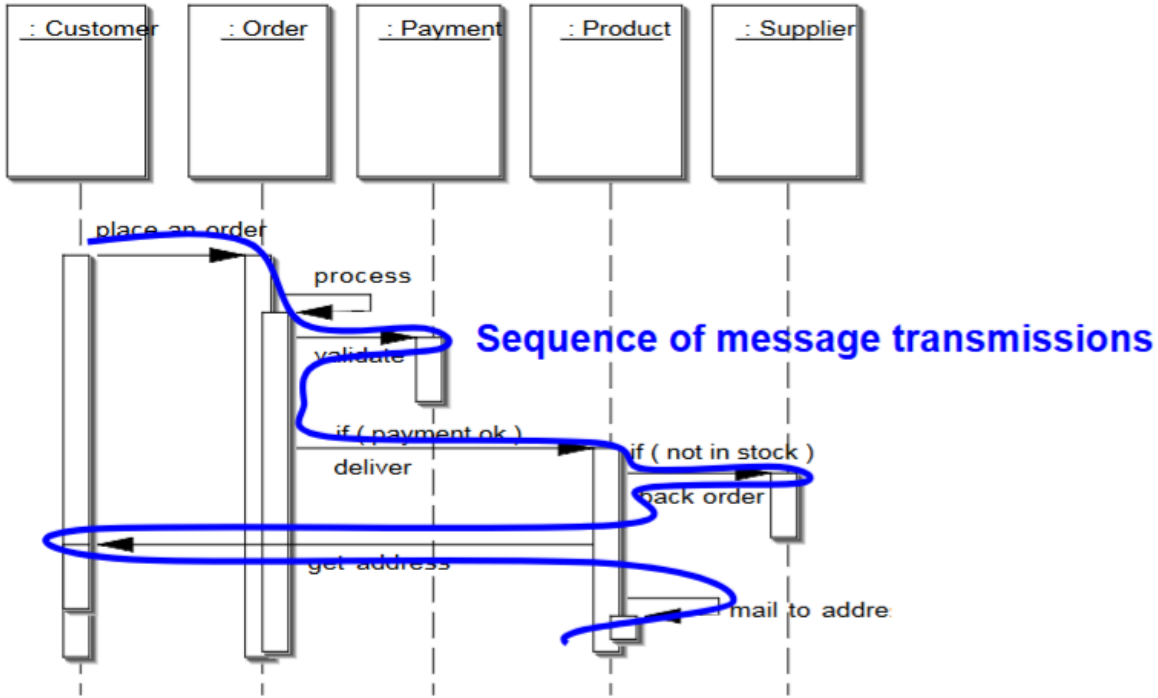


Sequence Diagrams · Main Entities

Example for decision using **ALT** frame

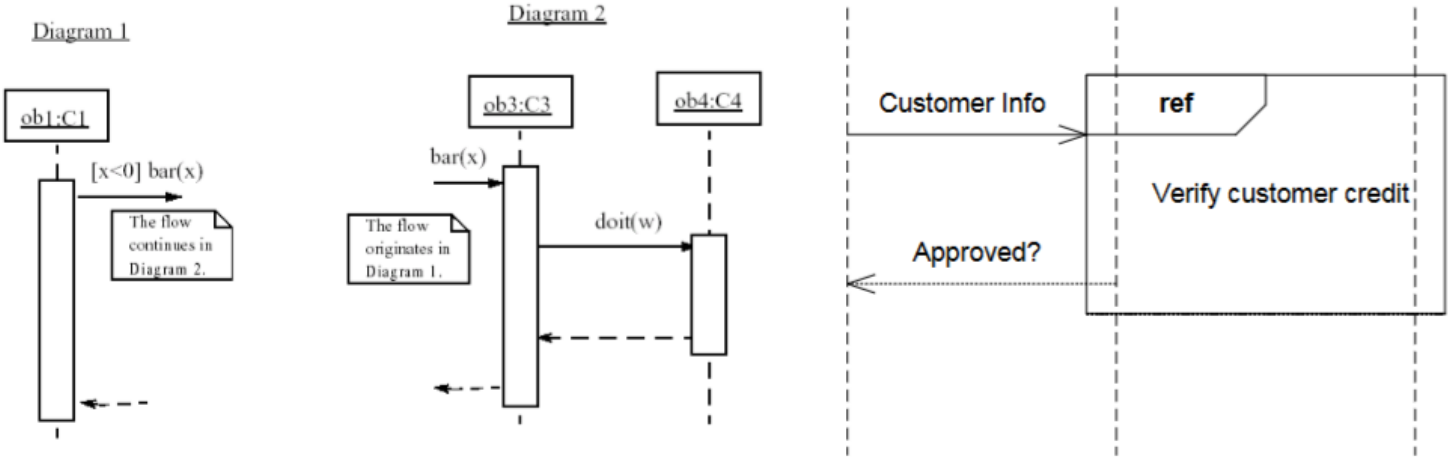


Sequence Diagrams : Flow

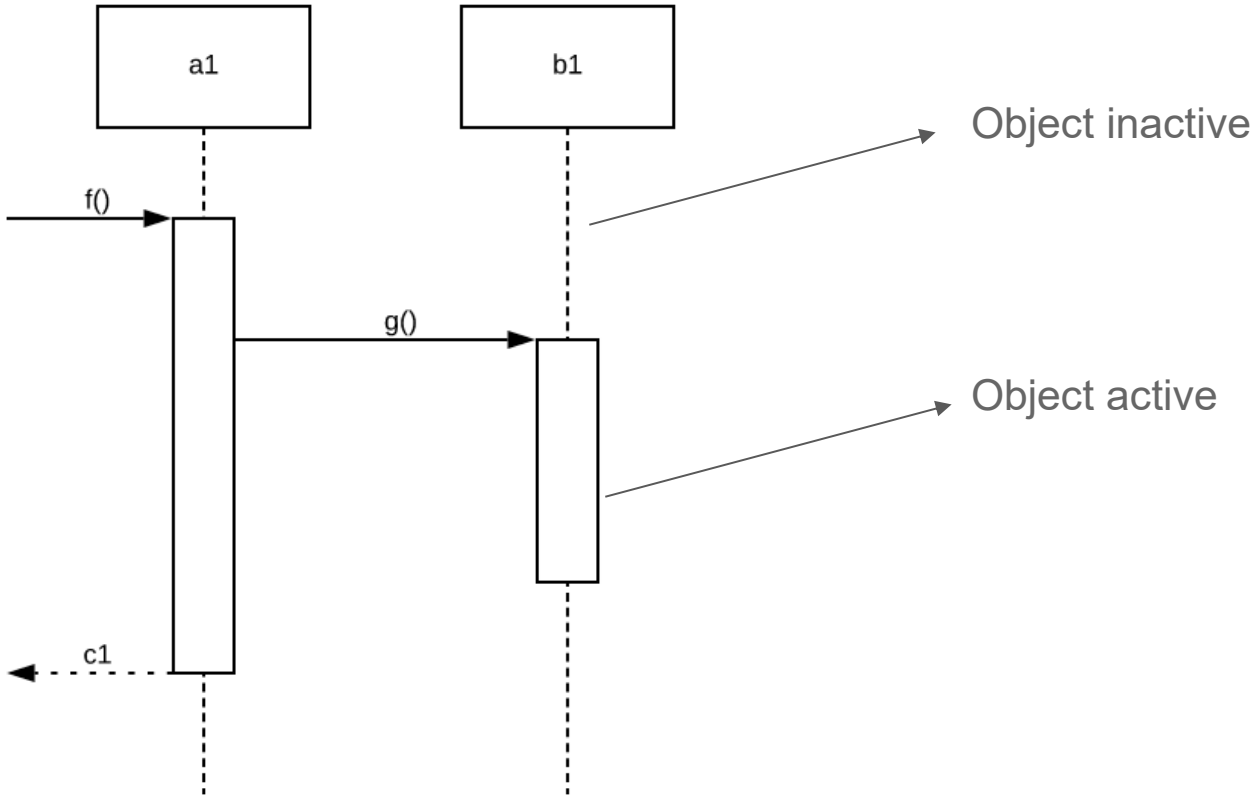


Sequence Diagrams : Linking Sequence Diagrams

- if a sequence diagram is too large or refers to another diagram, indicate it with either:
  - an unfinished arrow and a comment
  - a "`ref`" frame that names the other diagram

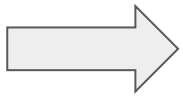


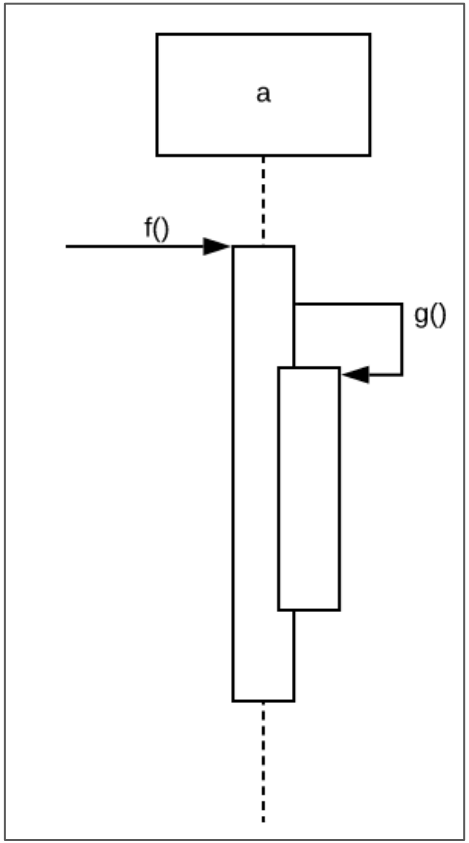
Example 1



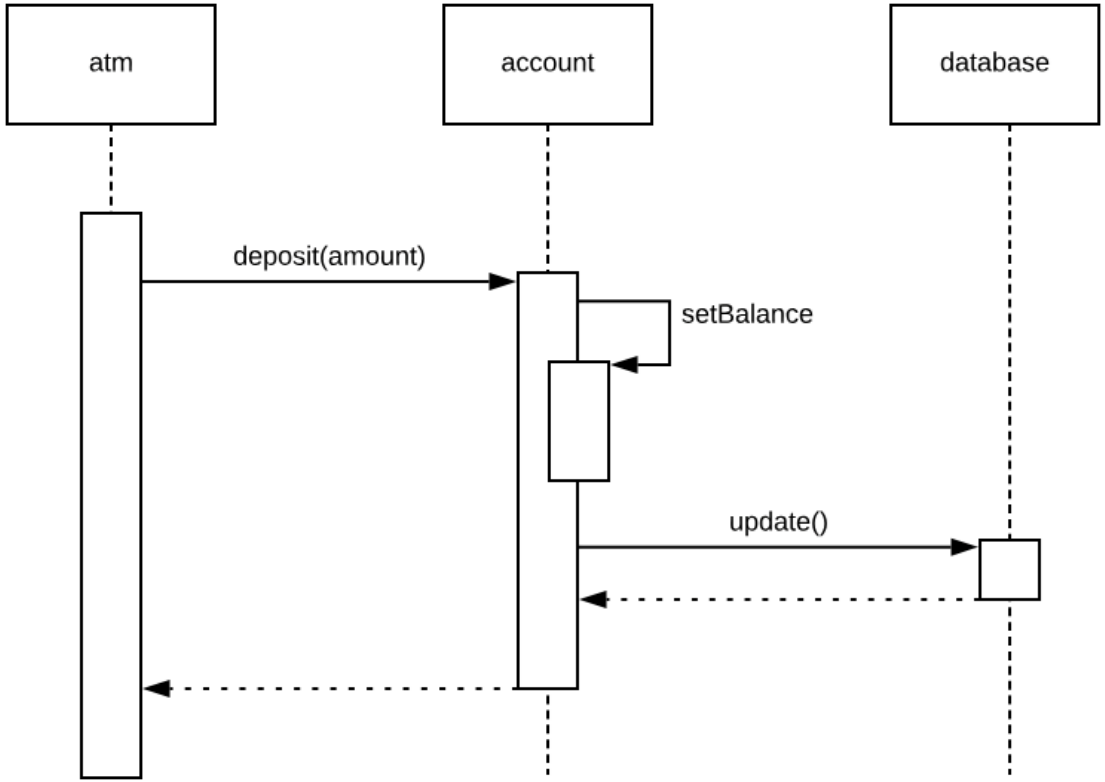
Example 2

```
class A {

  void g() {
    ...
  }

  void f() {
    ...
    g();
    ...
  }

  main() {
    A a = new A();
    a.f();
  }
}
```

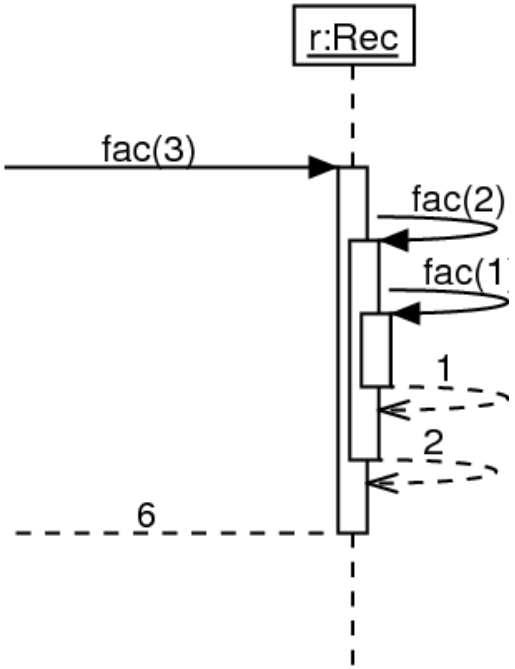




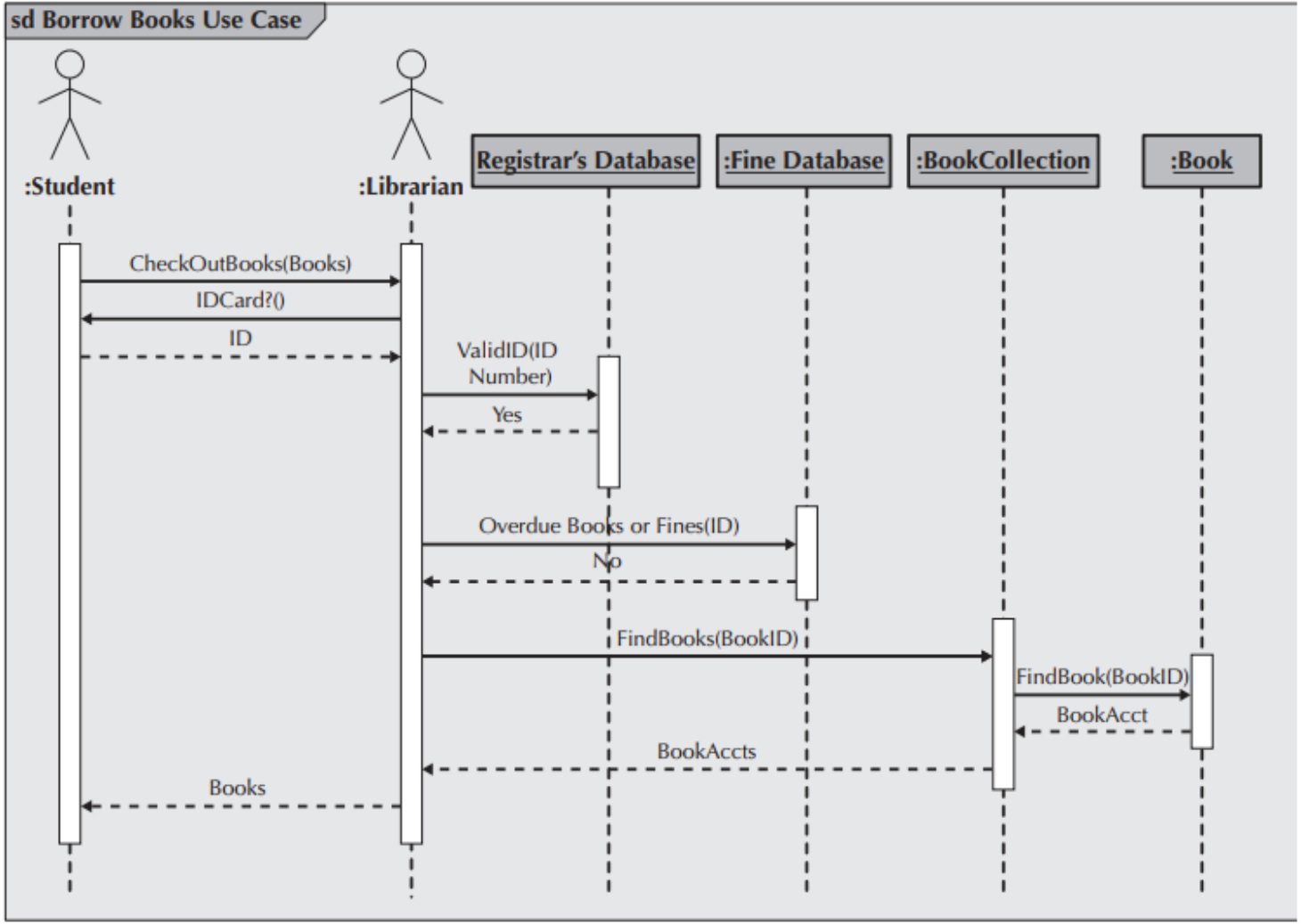
Example 3



Example 4

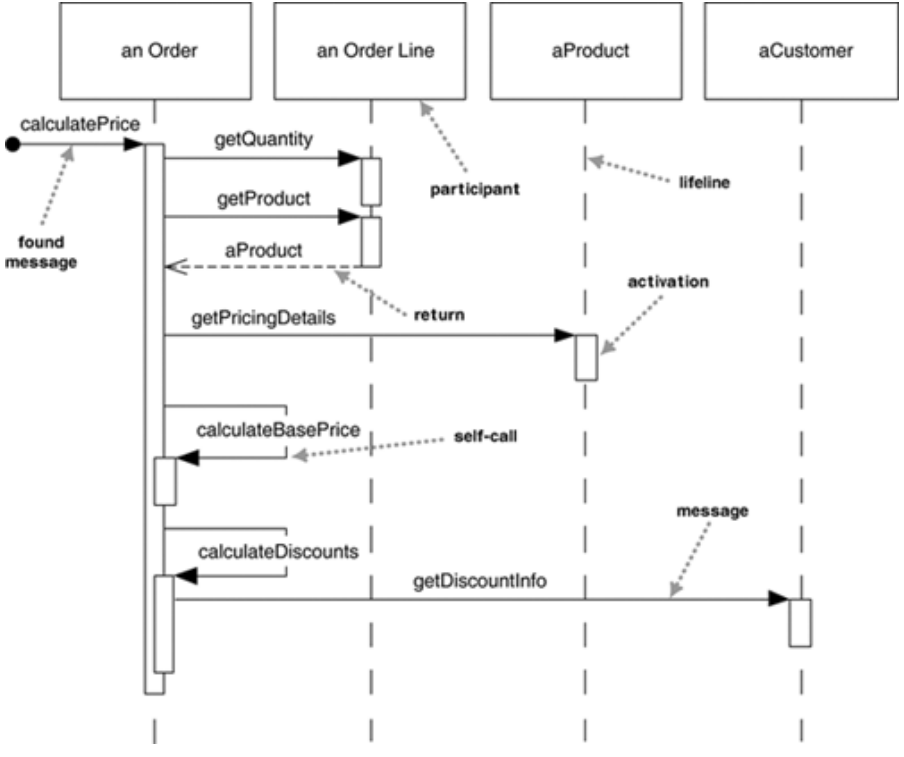


This is not a meaningful use of sequence diagrams

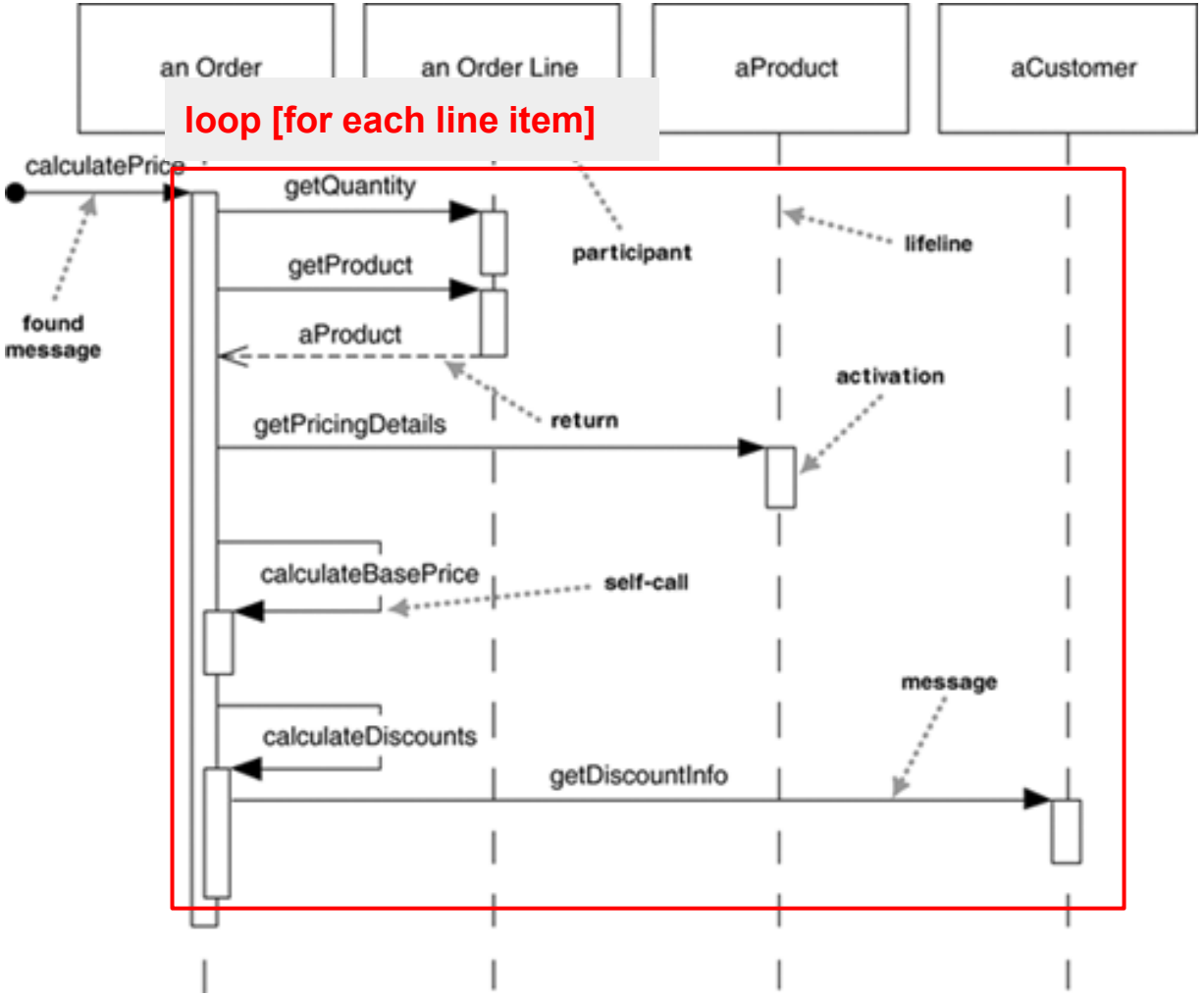

sd Borrow Books Use Case
:Student
:Librarian
Registrar's Database
:Fine Database
:BookCollection
:Book
CheckOutBooks(Books)
IDCard?()
ID
ValidID(ID Number)
Yes
Overdue Books or Fines(ID)
No
FindBooks(BookID)
FindBook(BookID)
BookAcct
BookAccts
Books

Exercises

This sequence diagram is intended to represent the method calls required to calculate the total value of an Order, composed of multiple OrderLines, each linked to a Product and a quantity. Analyze why the diagram fails to represent this process accurately.



Answer: Corrected diagram

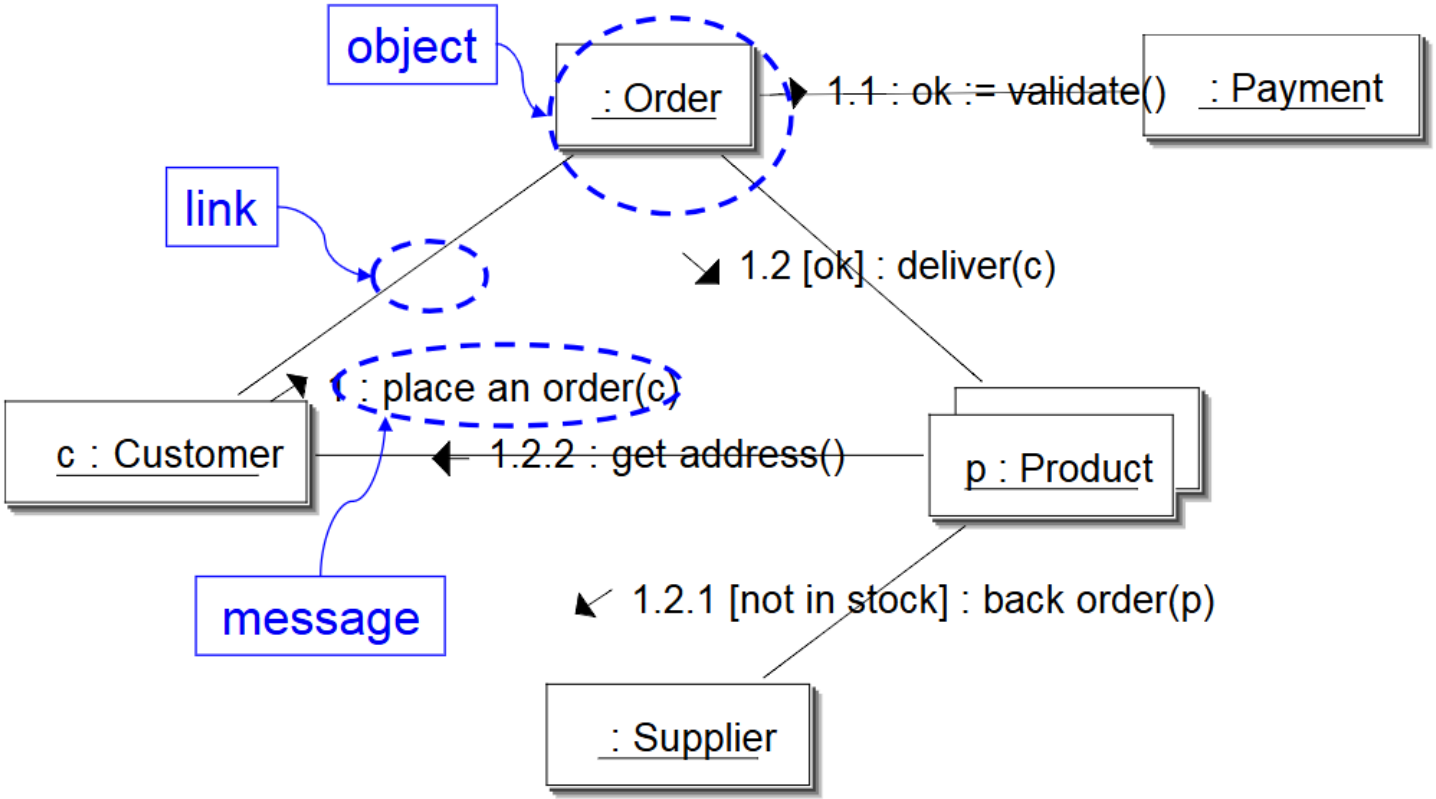


Communication (Collaboration) Diagram

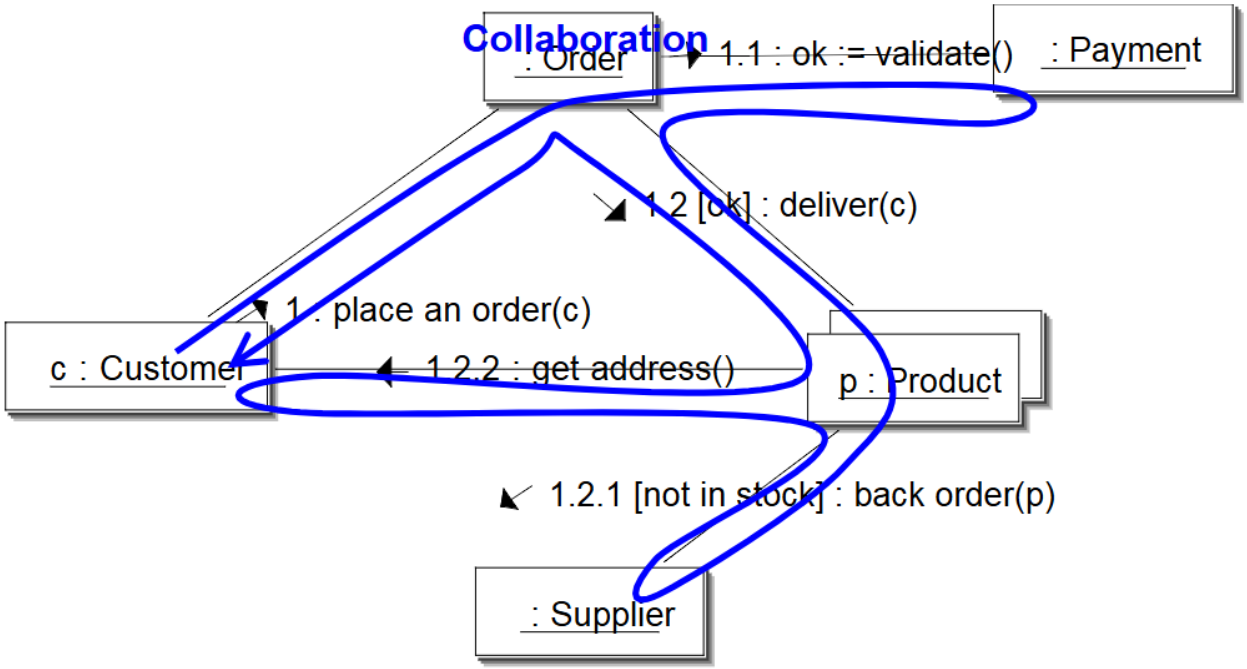
Communication (collaboration) diagram

- Communication diagrams show the message flow between objects in an application
- They also show implicitly the basic associations between classes
- Communication diagrams are drawn in the same way as sequence diagrams (and can be semantically equivalent to them)

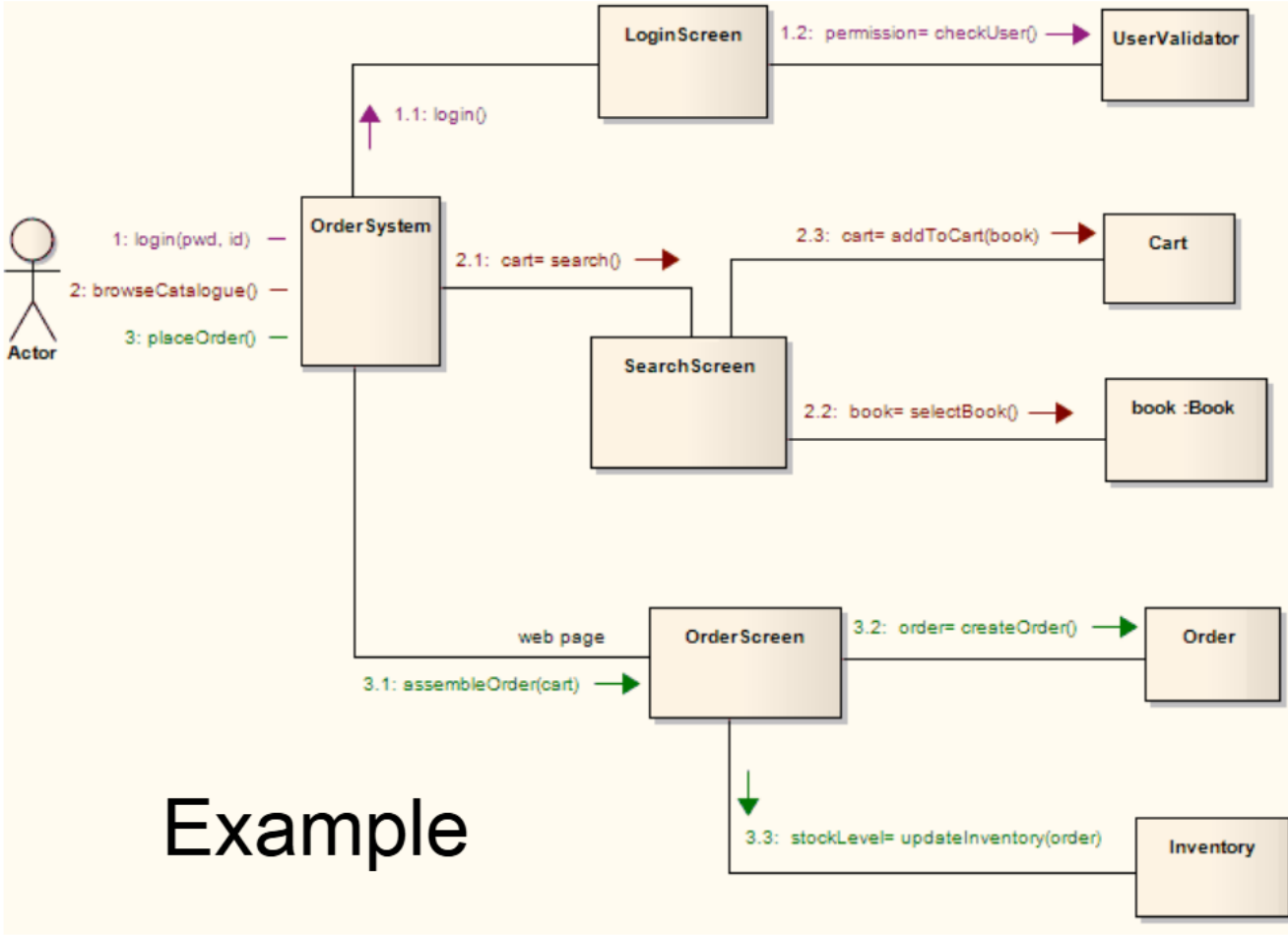
Communication diagram

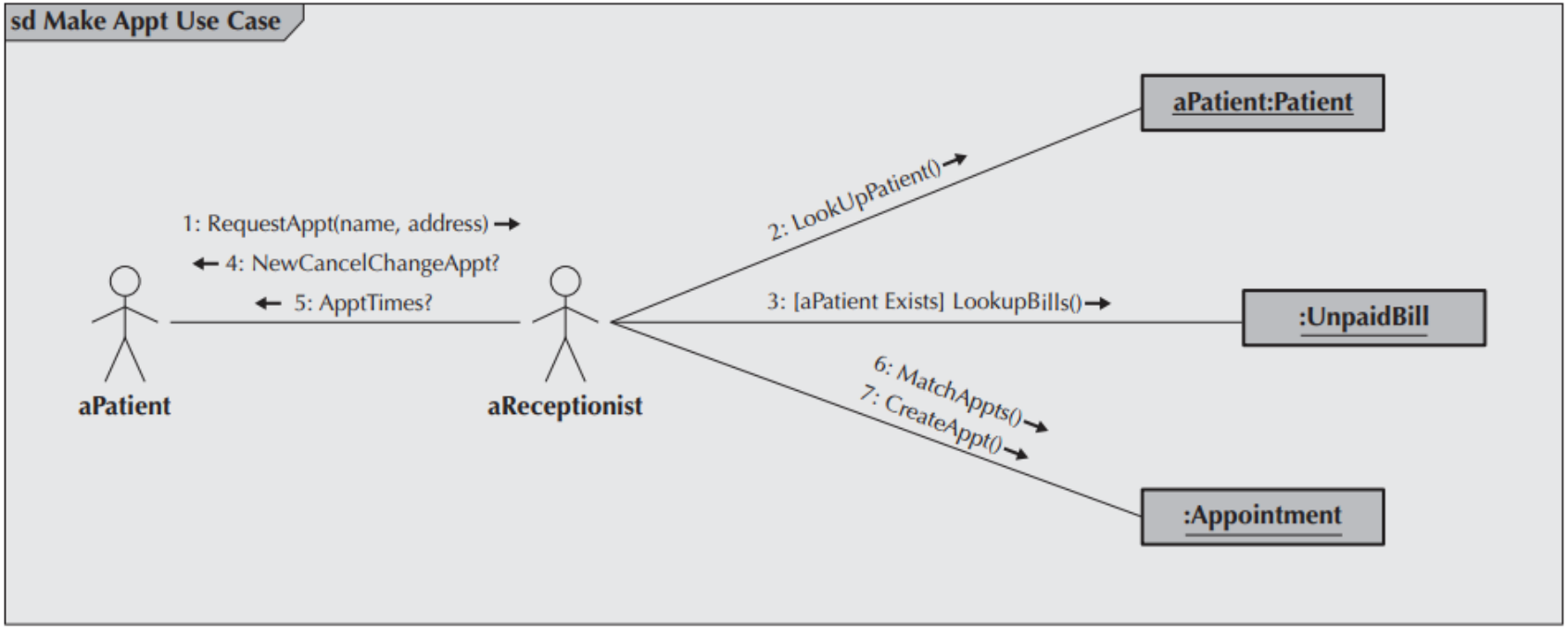


Communication diagram

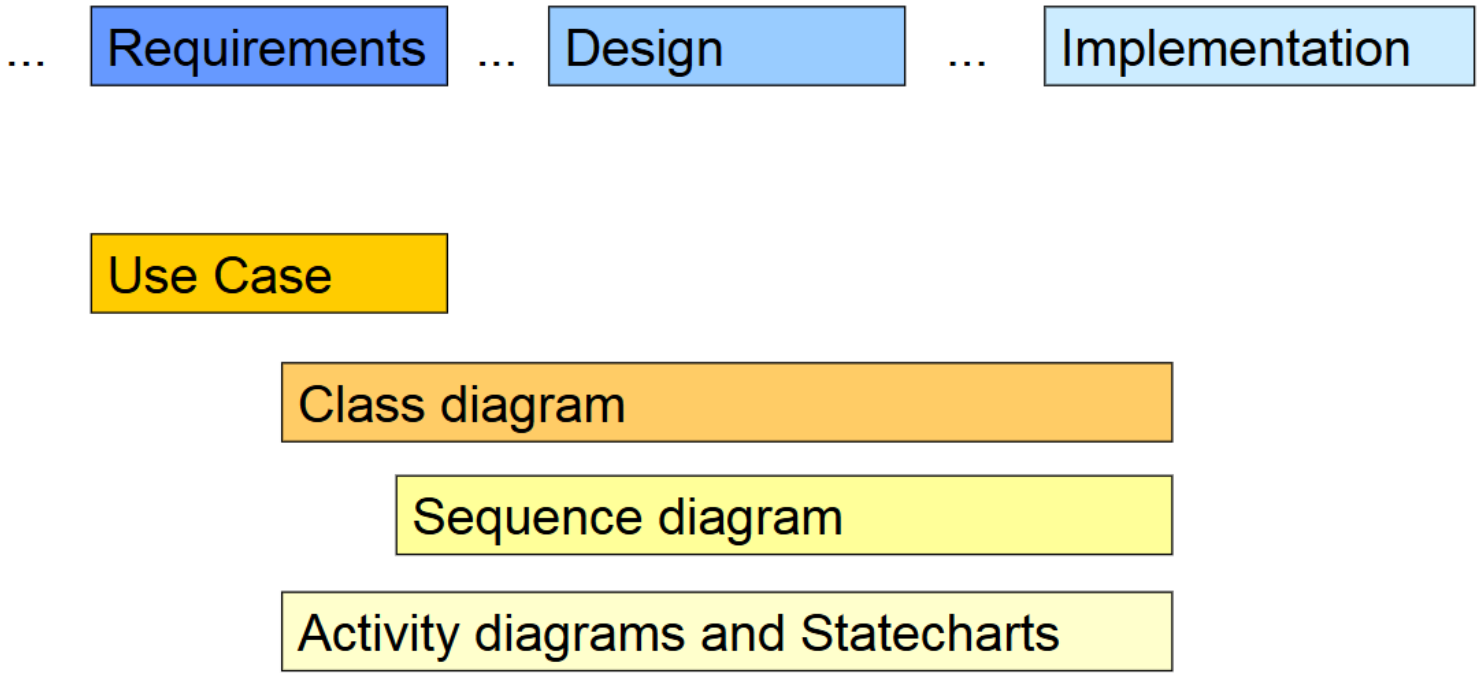


Communication diagram




sd Make Appt Use Case
aPatient:Patient
1: RequestAppt(name, address)
4: NewCancelChangeAppt?
5: ApptTimes?
2: LookUpPatient()
3: [aPatient Exists] LookupBills()
:UnpaidBill
6: MatchAppts()
7: CreateAppt()
aPatient
aReceptionist
:Appointment

Diagrams in lifecycle



Credit

- Portions of this presentation are derived from these courses:
  - Software Engineering course, ENSIA 2024/2025
  - UML basics, Paolo Ciancarini, university of Bologna,

End